Software Engineering (Sessional)

Course: CSE-356



CUET Online Stationery: Simplifying Access to Stationery for Every CUET Student

Team Members

Dipannita Paul Orni

ID: 2104125

Umme Sanjida

ID: 2104126

Oarisa Rebayet

ID: 2104129

Department of Computer Science & Engineering

Chittagong University of Engineering & Technology

Chattogram- 4349, Bangladesh

Team Members

Photo	Name	ID	Email	Contact No.	Total Credit Passed	CGPA
	Dipannita Paul Orni	2104125	u2104125@student.cuet.ac.bd	01818366549	101.75	
	Umme Sanjida	2104126	u2104126@student.cuet.ac.bd	01558910989	101.75	
	Oarisa Rebayet	2104129	u2104129@student.cuet.ac.bd	01575163718	101.75	

Executive Summary

The CUET Online Stationery System is a web-based platform developed to provide CUET students with convenient access to stationery and academic supplies within the university campus. The system reduces the need for off-campus purchases by allowing students to browse products, place orders, and receive items efficiently through a centralized online service.

The platform offers essential features such as a categorized product catalog, search and filter options, shopping cart and checkout facilities, multiple payment methods, and real-time order tracking. An administrative dashboard enables vendors and administrators to manage inventory, process orders, and monitor deliveries effectively. Role-based access control ensures secure and appropriate system usage for students and vendors.

The system follows a client-server architectural model with a centralized database to maintain data consistency and scalability. It is implemented using Next.js and MongoDB, while JIRA is used for project planning, task management, and team coordination throughout the development process.

By automating order handling and inventory management, the CUET Online Stationery System improves operational efficiency and supports CUET's vision of a smart, student-oriented campus. Future enhancements include mobile application development and extended automation features to further improve system usability and performance.

GitHub Repository:

Table of Contents

Contents

List of Figures **7**

List of Tables **7**

1 Introduction **8**

1.1 Goals and Objectives of the Project 8

1.2 Scope of the Work 9

1.2.1 Current Situation and Context 9

1.2.2 Competing Products (Available in the Market) 9

1.3 System Overview 9

1.4 Structure of the Document 10

2 Project Management Plan **11**

2.1 Project Organization 11

2.1.1 Individual Contribution to the Project 11

2.2 Process Model Used 11

2.2.1 Rationale for Choosing the Process Model 11

2.3 Risk Analysis 12

2.4 Constraints to Project Implementation 13

2.5 Hardware and Software Resource Requirements 14

2.6 Project Timeline and Schedule 14

2.7 Estimated Budget 15

2.8 Social, Cultural, and Environmental Impact of the Project 15

3 Requirement Specifications **16**

3.1 Stakeholders of the System 16

3.2 Use Case Diagram – Graphical and Textual Description 16

3.3 Development of Software Requirements Specification (SRS) 18

3.3.1 Software Requirements Specification (SRS) for Customer Module of CUET Online Stationery System 18

3.3.2 Software Requirements Specification (SRS) for Vendor Module of CUET Online Stationery System 23

3.4 Development of Use Case Template 26
3.4.1 Customer Use Case Template 27
3.4.2 Vendor Use Case Template 31
3.5 Activity Diagram 34
3.6 Static Model – Class Diagram 35
3.7 Dynamic Model – Sequence Diagram 36
3.8 Safety and Security Requirements 37
3.8.1 Access Requirements 37
3.8.2 Integrity Requirements 38
3.8.3 Privacy Requirements 38

4 Architectural Design **39**
4.1 Architectural Model/Style Used 39
4.2 Rationale for Choosing the Architectural Model/Style 40

5 Design **40**
5.1 Structured Chart Analysis 40
5.2 Component Level Design 41
5.3 Technical Design Document 42
5.3.1 Automation (Cronjobs, Queue, etc.) 42
5.3.2 Queue System (Asynchronous Processing) 43
5.4 GUI (Graphical User Interface) Design 43
5.4.1 Home Page and Login 44
5.4.2 Student Interface 45
5.4.3 Admin Interface 46
5.5 Data Flow Diagram 47
5.5.1 First Level DFD of the System 47
5.5.2 User DFD 48
5.5.3 Admin DFD 49
5.5.4 API Documentation 49
5.5.5 Swagger UI 54
5.5.6 Endpoints in Swagger 55
5.6 ER Diagram 57
5.7 Database Design Model and Script 59
5.7.1 Collection Descriptions 59

5.7.2 Entity Relationships 61
5.7.3 Mongoose Schema Implementation 61

6 Testing **69**

6.1 Manual Testing 69
6.2 Automated Testing 70
6.2.1 Automated Test Cases 71
6.2.2 Automated Test Execution Summary 72
6.2.3 Jenkins CI/CD Integration 72
6.3 Techniques Used for Test Generation 73
6.4 Assessment of the Goodness of the Test Suite 74
6.5 Sustainability Plan 74
6.5.1 Scalability 74
6.5.2 Flexibility and Customization 75

List of Figures

1 Customer Use Case Diagram 17
2 Vendor Use Case Diagram 18
3 Activity Diagram 35
4 Class Diagram 36
5 Sequence Diagram for Login and Product Search 37
6 Sequence Diagram for Order Placement and Payment 37
7 Data-Centered Architectural Model of CUET Online Stationery System . . 39
8 Structure Chart for CUET Online Stationery System 41
9 Component Level Design Diagram of Customer Service of CUET Online Stationery System 41
10 Home Page - Main Landing Page 44
11 Login Page - User Authentication 44
12 Student Dashboard - Personal Overview 45
13 Shopping Cart - Order Management 45
14 AI Recommendations - Smart Suggestions 45
15 Order History - Purchase Tracking 46
16 Admin Dashboard - System Overview 46
17 Order Management - Admin View 47

18 Product Management - Add/Edit Items 47
19 First Level DFD - Context Diagram of CUET Online Stationery System . 48
20 User DFD - Student Perspective 48
21 Admin DFD - Vendor/Admin Perspective 49
22 Swagger API Documentation Interface 54
23 Swagger UI for POST /auth/signup 55
24 Swagger UI for POST /auth/login 55
25 Swagger UI for GET /products 56
26 Swagger UI for POST /orders 56
27 Swagger UI for PUT /admin/products/id 56
28 Swagger UI for POST /feedback 57
29 Swagger UI for DELETE /admin/products/id 57
30 Entity Relationship Mapping (Cardinality View) 57
31 ER Diagram for CUET Online Stationery System 58
32 Jenkins Build and Test Execution - Successful Build 73

List of Tables

1 Individual Contribution to the Project 11
2 Risk Analysis of CUET Online Stationery 13
3 Project Implementation Plan 15
4 Estimated Budget for CUET Online Stationery System 15
5 Database Entity Relationships 61
6 Test Case - Add Product to Cart 70
7 Test Case - Vendor Add New Product 70
8 Automated Test Cases - Authentication Module 71
9 Automated Test Cases - Shopping Cart Module 71
10 Automated Test Cases - Orders Module 72
11 Automated Test Cases - Products Module 72
12 Automated Test Execution Summary 72

1 Introduction

In today's fast-paced academic environment, students often face challenges in accessing essential study materials conveniently and efficiently. CUET students, in particular, frequently rely on nearby stationery shops or make off-campus trips to purchase pens, notebooks, lab materials, and other academic supplies. Such trips can be time-consuming and inconvenient, especially during exam periods, late-night study sessions, or unfavorable weather conditions.

To address these challenges, the **CUET Online Stationery** platform has been proposed as a comprehensive digital solution. This system allows students to browse a wide range of stationery items, place orders, and receive their supplies directly at campus halls or designated pickup points. By integrating features such as an intuitive product catalog, easy payment options through bKash or Cash on Delivery (COD), order tracking, and an AI-driven recommendation system, the platform aims to streamline the entire process of acquiring academic materials.

1.1 Goals and Objectives of the Project

The main goal of the **CUET Online Stationery** system is to provide CUET students with a convenient and time-saving digital platform for purchasing stationery and academic supplies directly from campus. The project aims to enhance accessibility and promote a smart campus environment.

Objectives

- To provide a time-saving solution for purchasing stationery within CUET.
- To ensure students have quick access to essential academic supplies at all times.
- To simplify order management, inventory control, and delivery processes for the admin.
- To implement AI-driven item recommendations based on student needs.
- To offer secure payment options through bKash or Cash on Delivery.

1.2 Scope of the Work

The project focuses on developing an online platform where CUET students can order stationery items such as pens, notebooks, lab materials, and art supplies. Orders will be processed and delivered to campus halls or designated pickup points. The system supports both student-facing and admin-facing interfaces for smooth operation and management.

1.2.1 Current Situation and Context

Currently, CUET students depend on nearby stationery shops or off-campus trips to purchase essential items. This process becomes inconvenient during exam periods, late-night study sessions, or unfavorable weather conditions. The proposed system addresses these issues by creating a campus-centric e-commerce platform that allows students to browse, order, and receive stationery efficiently within the university premises. This aligns with CUET's mission to build a technologically advanced and student-friendly learning environment.

1.2.2 Competing Products (Available in the Market)

There are both direct and indirect competitors in the market:

Direct Competitors:

- StationeryBD
- Rokomari Stationery

Indirect Competitors:

- Local stationery shops near CUET that provide immediate purchase options but lack online ordering and delivery convenience.

1.3 System Overview

The **CUET Online Stationery** platform enables students to order essential academic supplies through a user-friendly web interface. It provides features such as:

- Product catalog with categorized listings and images.
- Search and filter functionality for quick navigation.
- Shopping cart and checkout system with multiple payment options.

- Real-time order tracking and delivery management.
- Admin dashboard for managing inventory, products, and orders.
- AI-based smart recommendations and discount systems.

The system ensures smooth interaction between students and admins and promotes efficient resource management within the CUET campus.

1.4 Structure of the Document

This report is organized into several chapters, each describing a specific aspect of the CUET Online Stationery System, from project planning to system testing. The structure of the document is outlined below:

Chapter 2: Project Management Plan

This chapter describes the organizational structure of the project and the development approach followed. It includes the selected software development life cycle model, individual team contributions, risk analysis, implementation constraints, required hardware and software resources, project timeline, estimated budget, and an evaluation of the social, cultural, and environmental impact of the system.

Chapter 3: Requirement Specifications

This chapter presents the detailed requirement analysis of the CUET Online Stationery System. It identifies system stakeholders and provides graphical and textual representations of use cases. The chapter also includes the Software Requirements Specification (SRS) for both customer and vendor modules, along with activity diagrams, class diagrams, sequence diagrams, and safety, security, integrity, and privacy requirements.

Chapter 4: Architecture

This chapter explains the architectural model adopted for the system. It discusses the selected architectural style, the rationale behind choosing it, and the interaction between different system components, highlighting how centralized data management and client-server communication are achieved.

Chapter 5: Design

This chapter focuses on the detailed design of the system. It includes structured chart analysis, component-level design, technical design documentation, database design, entity–relationship diagrams, and API-related documentation. The chapter illustrates how system requirements are transformed into a concrete and implementable design.

Chapter 6: Testing

This chapter outlines the testing strategies applied to verify and validate the system. It covers both manual and automated testing approaches, test case descriptions, test execution summaries, and continuous integration support to ensure system reliability, correctness, and overall performance.

2 Project Management Plan

2.1 Project Organization

The CUET Online Stationery project is developed by a three member team working collaboratively to design, develop, and test a web-based application for CUET students. The team members coordinate through weekly meetings to review progress and assign tasks, ensuring each phase of development from design to deployment follows a structured workflow.

2.1.1 Individual Contribution to the Project

Member Name	Requirement Specification	Planning	Designing	Front End	Back End	Testing
Dipannita Paul Orni	✓	✓	✓		✓	✓
Umme Sanjida	✓	✓	✓	✓		
Oarisa Rebayet	✓	✓	✓	✓	✓	✓

Table 1: Individual Contribution to the Project

2.2 Process Model Used

The project follows a Prototyping Process Model integrated with Agile principles. This hybrid approach is effective for projects where requirements evolve through user interaction and continuous feedback. In this model, an initial prototype of the system is developed to demonstrate major functionalities. The prototype is evaluated by users and stakeholders to gather feedback, which is used to refine the system through successive iterations until the final version is achieved.

2.2.1 Rationale for Choosing the Process Model

The Agile-based Prototyping Model was selected for the CUET Online Stationery project because it aligns with the project's objectives, team size, and academic timeline. The following points highlight the key reasons behind this selection:

- **User-Centered and Iterative Design:** The project emphasizes a user-friendly web interface. Early prototypes allow usability testing and feedback, ensuring the design remains intuitive and aligned with CUET students' expectations.
- **Early Feedback and Requirement Refinement:** Each prototype iteration involves collecting input from students and stakeholders. This feedback-driven cycle refines requirements and interface design, minimizing rework and improving accuracy.
- **Flexibility and Adaptability:** Integrating Agile principles allows the team to respond quickly to changing requirements or new feature ideas, such as AI-based recommendations or coupon systems, without disrupting progress.
- **Reduced Development Risk:** Early evaluation of functional prototypes helps detect usability or technical issues before full-scale implementation, reducing risks and improving overall quality.
- **Efficient Use of Time and Resources:** Given the short academic timeline and limited resources, this model enables rapid prototyping and testing of key modules without excessive documentation or lengthy planning phases.
- **Enhanced Team Collaboration:** The Agile structure encourages regular communication and coordination, allowing different modules (product catalog, cart system, admin dashboard) to be developed and tested concurrently.
- **Higher User Acceptance:** Continuous stakeholder involvement during prototype evaluation ensures the final system closely reflects user needs, resulting in greater satisfaction and adoption among CUET students.

2.3 Risk Analysis

Risk management is a vital part of the CUET Online Stationery project to ensure timely completion and user satisfaction. Risks may arise from various sources, including technical challenges, schedule constraints, operational issues, evolving user requirements, and product functionality. Early identification and mitigation of these risks help minimize delays and ensure that the final product meets the expectations of CUET students. By proactively addressing potential issues, the project team can enhance efficiency and deliver a reliable online stationery platform. The project risks are categorized as follows:

- **Technical Risk (TR):** Risks related to technology, coding, integration, and system failures.
- **Schedule Risk (SR):** Risks affecting project timelines and milestone completion.
- **User/Business Risk (UBR):** Risks impacting user satisfaction, usability, or changing requirements.
- **Product-Based Risk (PBR):** Risks associated with features, functionality, performance, and overall quality of the product.

Risks	**Category**	**Probability**	**Impact**
Team may face difficulty integrating frontend and backend	TR	35%	2
Delivery deadline may be delayed	SR	30%	2
Database errors may lead to data loss	TR	25%	3
New feature requests may arise late	UBR	50%	4
Security vulnerabilities may expose sensitive data	PBR	40%	2

Table 2: Risk Analysis of CUET Online Stationery

2.4 Constraints to Project Implementation

Several potential constraints may affect the implementation of the **CUET Online Stationery** project. These include:

- **Technical Limitations:** As the development relies on student-level resources and free-tier services, limited server capacity or data storage may restrict scalability.
- **Time Constraints:** The project must be completed within the academic semester, leaving a short time for testing and optimization.
- **Financial Constraints:** The team operates under a minimal budget, relying primarily on free or educational software licenses, which may limit access to premium tools or APIs.
- **Connectivity Issues:** Since the system depends on stable internet connectivity, network interruptions within the campus may affect ordering operations.
- **Human Resource Constraints:** The project is being developed by a small student team, which may slow down parallel progress in different modules such as frontend, backend, and testing.

2.5 Hardware and Software Resource Requirements

The project will be implemented using standard computing hardware and widely available software tools suitable for web development.

Hardware Requirements:

- Personal computers or laptops for development and testing.
- Internet connection for communication, hosting, and database operations.
- Mobile devices for testing responsive layouts and user experience.

Software Requirements:

- **Frontend & Backend Framework:** Next.js, a full-stack React framework with server-side rendering.
- **Database:** MongoDB (for authentication, data management, and storage).
- **Development Tools:** Visual Studio Code, Git/GitHub (for version control)
- **Project Management Tools:** JIRA, for tracking tasks, managing sprints and monitoring project progress in an Agile environment.
- **AI Recommendation System:** Python and Flask (for model training and deployment)

2.6 Project Timeline and Schedule

The **CUET Online Stationery** project will be executed within one academic semester. The tentative timeline is outlined below:

Week	Tasks/Activities
Week 1	Project planning, team meetings, and finalization of requirements.
Week 2–3	UI/UX design and setup of development environment.
Week 4–5	Frontend development (product catalog, search & filter, and cart system).
Week 6–7	Backend integration, database setup, and order management module.
Week 8	Implementation of admin dashboard and delivery management system.
Week 9	Integration of payment gateway and AI-based recommendation system.
Week 10	Testing, debugging, and user feedback collection.
Week 11	Final refinements, documentation, and project submission.

Table 3: Project Implementation Plan

2.7 Estimated Budget

The estimated budget for the CUET Online Stationery System project is summarized below. Since this is a course-based project, we only account for the hardware used by the team and other miscellaneous costs.

Item	Estimated Cost (BDT)
Hardware (3 Laptops/PCs for development/testing)	$3 \times 70{,}000 = 210{,}000$
Front-end Development Tools	0 (Open-source)
Back-end Development Tools	0 (Open-source)
Miscellaneous (Utilities, Internet, Maintenance)	3,000
Total Estimated Budget	**213,000**

Table 4: Estimated Budget for CUET Online Stationery System

2.8 Social, Cultural, and Environmental Impact of the Project

The **CUET Online Stationery** project contributes positively to CUET's social and academic ecosystem in several ways:

- **Social Impact:** The platform makes life easier for students by providing quick to stationery, especially during busy exam periods or late-night study sessions. It helps students save time and encourages them to embrace digital tools in their daily academic routine.

- **Cultural Impact:** By supporting a campus-based digital marketplace, the project encourages innovation and entrepreneurial thinking among students. It promotes a culture where technology is used wisely to solve everyday problems.
- **Environmental Impact:** By reducing the need for students to travel off-campus for stationery, the system helps lower overall transportation activity. Additionally, keeping digital orders and receipts minimizes paper waste, promoting more environmentally conscious habits within the university community.

3 Requirement Specifications

3.1 Stakeholders of the System

The stakeholders of the **CUET Online Stationery** system include all individuals and entities who are directly or indirectly involved in its development, operation, or usage. The primary stakeholders are described as follows:

- **Students:** The main end-users of the system. They can browse, order, and purchase stationery items through the platform. They are the primary beneficiaries of the convenience, accessibility, and time-saving features offered by the system.
- **Admins/Vendors:** Responsible for managing the platform's operations. Their duties include updating the product catalog, monitoring inventory levels, processing orders, managing deliveries, and handling customer feedback.
- **System Developers:** The project team responsible for designing, developing, testing, and maintaining the platform. They ensure that the system meets all functional and non-functional requirements while maintaining security and performance standards.

Collectively, these stakeholders contribute to the system's development, operation, and sustainability, ensuring that it serves the CUET community effectively and reliably.

3.2 Use Case Diagram – Graphical and Textual Description

The use case diagram illustrates the functional requirements and interactions within the online stationery store system. It uses actors and use cases to represent the system's

behavior, where actors denote the users of the system, namely the Customer and the Vendor, and use cases represent the functionalities available to them.

Two separate use case diagrams have been developed to reflect the distinct roles and responsibilities of the system users. One diagram focuses on the Customer, highlighting activities such as browsing products, placing orders, making payments, and tracking order status. The other diagram represents the Vendor, showing functionalities related to product management, inventory monitoring, order processing, and delivery updates.

Figure 1 presents the Customer use case diagram, and Figure 2 presents the Vendor use case diagram. Together, these diagrams provide a clear overview of how different users interact with the system and serve as a basis for requirement analysis, system design, and further development.

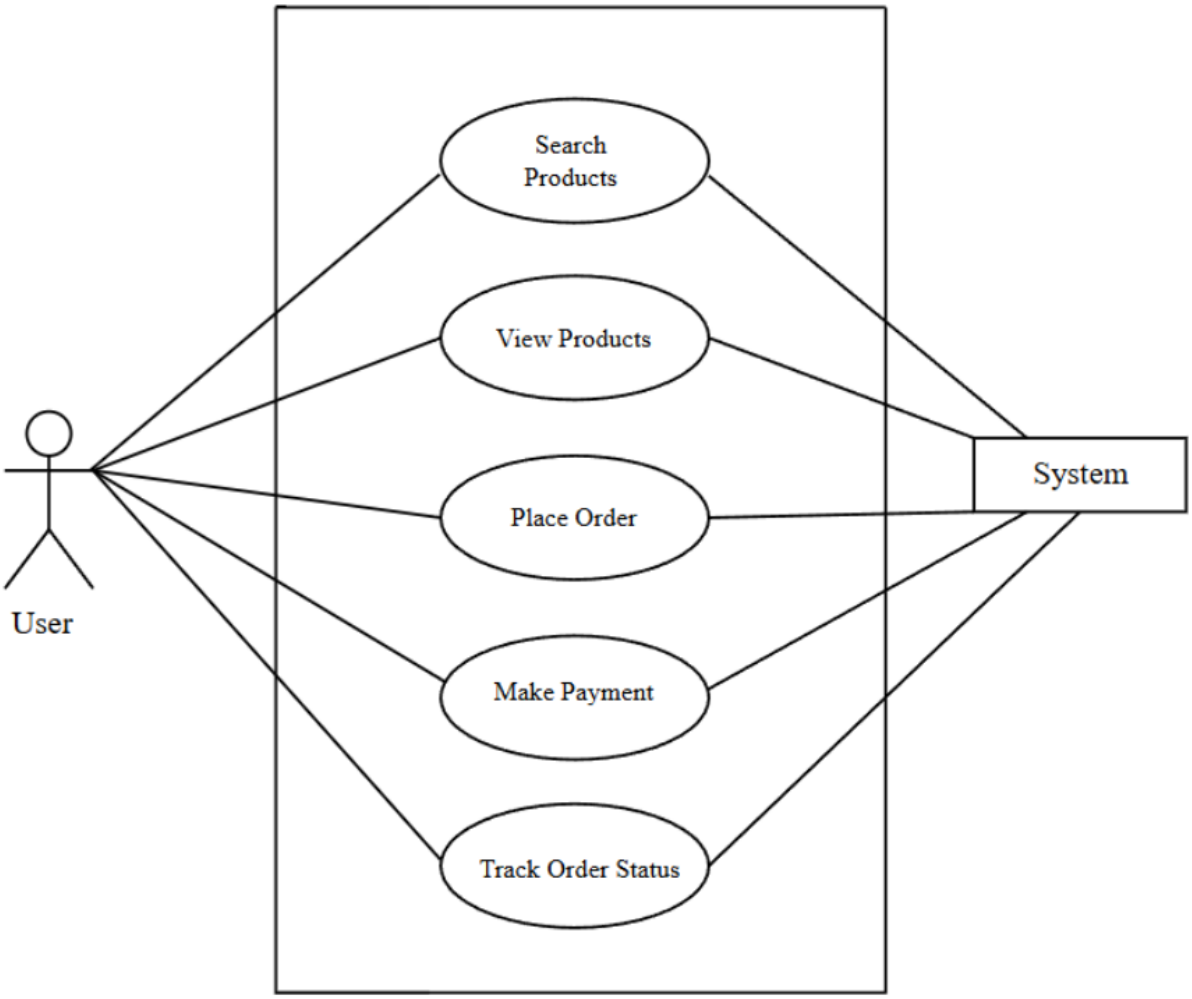


Figure 1: Customer Use Case Diagram

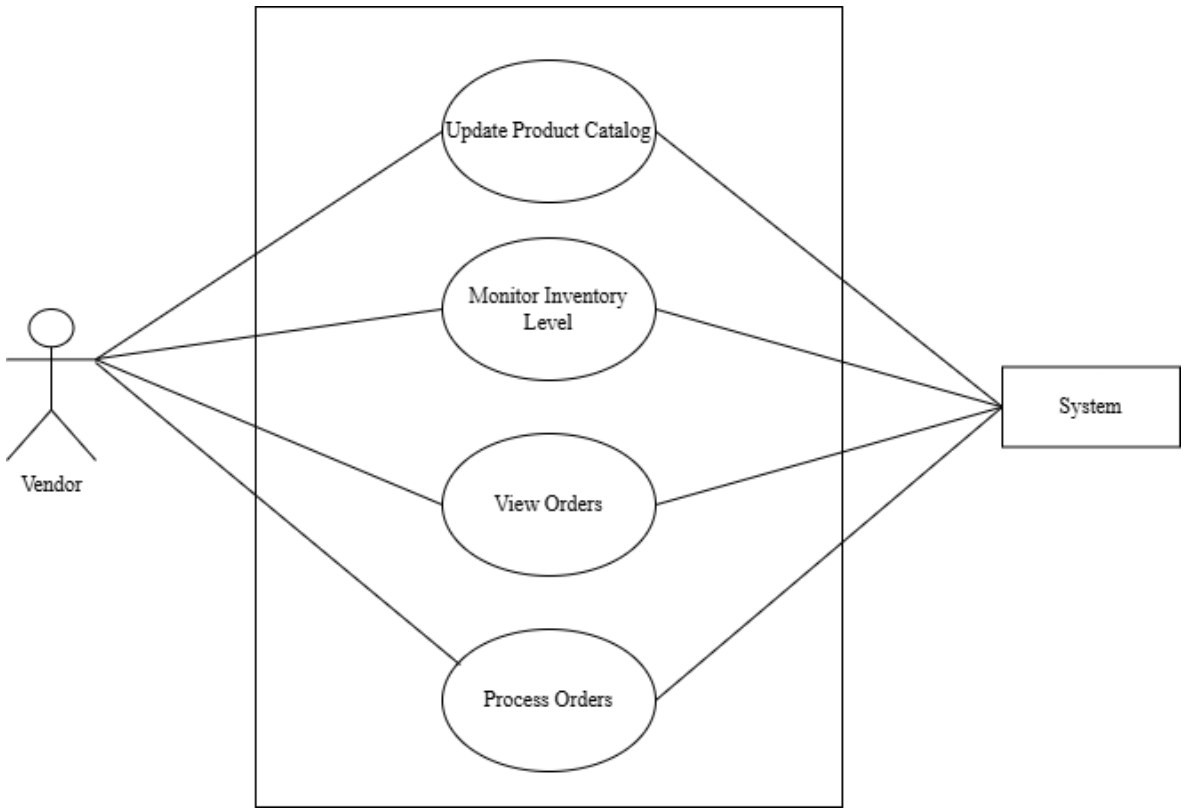


Figure 2: Vendor Use Case Diagram

3.3 Development of Software Requirements Specification (SRS)

The development of the Software Requirements Specification (SRS) for the CUET Online Stationery system was carried out through a systematic process aimed at clearly defining all functional and non-functional requirements of the project. The SRS serves as a formal document that bridges the gap between the users' needs and the technical implementation by providing a detailed description of what the system is expected to do.

3.3.1 Software Requirements Specification (SRS) for Customer Module of CUET Online Stationery System

1. Introduction

Purpose:

The purpose of this document is to define the software requirements for the **Customer Module** of the CUET Online Stationery System. This module allows CUET students to browse products, place orders, make payments, and track their delivery status conveniently from within the campus.

Scope:

The Customer Module is designed to serve CUET students by providing a web-based

platform for purchasing stationery items such as notebooks, pens, art materials, and lab supplies. The system supports product browsing, cart management, secure payment, and order tracking. It ensures faster service, improved convenience, and minimizes the need to visit physical stores.

2. Overall Description

Product Perspective:

The Customer Module is part of the *CUET Online Stationery* web application. It acts as the frontend interface for students, while the Vendor Module manages backend operations such as inventory and delivery. The system uses a central database to store user accounts, product details, and order information.

User Classes and Characteristics:

- **Student (Customer):** Can browse products, search items, add to cart, make payments, and track order status.

3. Operating Environment

The system will be web-based and accessible through standard web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. It will be built using **Next.js**, a full-stack React framework that handles both frontend rendering and backend API management through its built-in server-side capabilities.

The database environment will use **MongoDB**, providing efficient handling of user authentication, product data, and transaction records. All data operations will occur over a secure connection using **HTTPS**.

The system will be hosted on a cloud-based platform compatible with **Next.js deployment** (such as Vercel or Render), ensuring high availability and scalability. The application will be compatible with both desktop and mobile devices, ensuring responsive design and smooth interaction across various screen sizes and operating systems.

4. Functional Requirements

Search Products:

- The system shall allow customers to search for stationery items by name, brand, or category.
- Search results shall display item name, price, image, and stock status.

View Product Catalog:

- The system shall display the product catalog categorized into notebooks, pens, lab supplies, and art materials.
- Customers shall be able to sort or filter products by price, rating, or brand.

Place Order:

- Customers shall be able to select multiple items and add them to the shopping cart.
- The system shall validate item availability before checkout.
- Customers shall receive a detailed summary before confirming the order.

Make Payment:

- The system shall provide two payment methods: Online transaction and Cash on Delivery (COD).
- The payment gateway shall confirm successful transactions and record payment status.

Track Order Status:

- Customers shall be able to view their order status: *Pending, Packed, Delivered.*
- The system shall notify the customer of each status update via the user dashboard.

5. Non-Functional Requirements

The following non-functional requirements define the overall qualities and constraints of the Customer Module:

1. **Performance Requirements:**
    - The system should display the homepage and product catalog within 3 seconds under normal network conditions.
    - It should handle up to 100 concurrent users without system slowdown.
    - Search and checkout operations must respond within 5 seconds.
2. **Usability Requirements:**
    - The interface must be intuitive, minimal, and suitable for both desktop and mobile users.

- Navigation menus, filters, and search bars should be easy to use.
- Error messages must be clear and descriptive to guide users during failures (e.g., payment errors).

3. **Reliability Requirements:**
   - The system must maintain at least 99% uptime during academic hours.
   - Order and payment data must remain intact even in case of temporary system downtime.
   - A backup of the MongoDB database should occur automatically every 24 hours.
4. **Security Requirements:**
   - Sensitive data such as passwords and payment information must be encrypted using secure hashing algorithms.
   - Authentication must be implemented using JWT (JSON Web Tokens) or OAuth.
   - All transactions must be performed over HTTPS to ensure data integrity and confidentiality.
5. **Maintainability and Scalability:**
   - The modular architecture of Next.js and Node.js should allow for easy addition of new features.
   - Code should follow consistent naming and documentation standards for maintainability.
   - The system must support scaling to accommodate more users and vendors in the future.

6. Constraints

- The application must be developed using open-source technologies such as Next.js and MongoDB.
- Continuous internet connectivity is required for browsing, authentication, and order placement.
- The system must comply with CUET's IT and data security policies.

- The project is developed under limited academic resources and must be completed within the semester timeframe.
- Hosting is limited to university infrastructure or available free-tier cloud services.

7. Assumptions and Dependencies

- Students have access to devices with modern web browsers and stable internet connections.
- Vendors and admins will update inventory and product data regularly.
- MongoDB Atlas and Next.js server will remain functional and accessible.
- Delivery operations depend on on-campus logistics and delivery personnel availability.

8. Future Enhancements

1. **Discount and Loyalty Features:**
   Introduce coupons, discounts, and a reward points system for frequent buyers.
2. **Real-Time Order Tracking:**
   Implement live delivery tracking integrated with GPS or on-campus map updates.
3. **Multi-Vendor Support:**
   Extend the platform to include multiple campus vendors with independent dashboards.
4. **Mobile Application:**
   Develop a cross-platform mobile app (React Native) with offline browsing and push notifications.

9. Conclusion

The Customer Module of the CUET Online Stationery System aims to streamline the purchasing experience for students, providing an efficient, secure, and scalable online platform to order academic and stationery supplies conveniently from anywhere on campus.

3.3.2 Software Requirements Specification (SRS) for Vendor Module of CUET Online Stationery System

1. Introduction

Purpose:

The purpose of this document is to define the software requirements for the **Vendor Module** of the CUET Online Stationery System. This module allows vendors or admins to manage the product catalog, monitor inventory, process orders, and update delivery statuses efficiently.

Scope:

The Vendor Module is designed to provide authorized personnel with a web-based dashboard to manage products, inventory, and student orders. It ensures real-time updates to the central database for accurate order fulfillment and inventory tracking, enabling efficient campus-wide stationery service management.

2. Overall Description

Product Perspective:

The Vendor Module is the administrative interface of the *CUET Online Stationery* web application. Vendors and admins manage stock, orders, and deliveries while the system ensures all updates reflect in real time for student access through the Customer Module. The module connects directly to the central database.

User Classes and Characteristics:

- **Vendor/Admin:** Authorized personnel responsible for managing products, monitoring inventory, processing orders, and updating delivery statuses.

3. Operating Environment

The system will be web-based and accessible through standard web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. It will be built using **Next.js**, a full-stack React framework that handles both frontend rendering and backend API management through its built-in server-side capabilities.

The database environment will use **MongoDB**, providing efficient handling of user authentication, product data, and transaction records. All data operations will occur over a secure connection using **HTTPS**.

The system will be hosted on a cloud-based platform compatible with **Next.js deployment** (such as Vercel or Render), ensuring high availability and scalability. The

application will be compatible with both desktop and mobile devices, ensuring responsive design and smooth interaction across various screen sizes and operating systems.

4. Functional Requirements

Update Product Catalog:

- Vendors shall be able to add, edit, or remove stationery products.
- Updates to products shall immediately reflect in the Customer Module catalog.

Monitor Inventory Levels:

- The system shall display product quantities in real time.
- Vendors shall receive alerts when stock falls below predefined thresholds.

View Orders:

- Vendors shall view all incoming orders, including customer details and ordered items.
- Orders shall be filterable by status: *Pending, Packed, Delivered*.

Process Orders:

- Vendors shall update order statuses during processing.
- When an order is marked as *Delivered*, a notification shall be sent to the customer.

5. Non-Functional Requirements

1. **Performance Requirements:**
    - Updates to inventory, product data, and order status must reflect in real time.
    - The admin dashboard should load within 3 seconds even with up to 500 products.
    - The system must handle simultaneous data requests from multiple vendors without performance degradation.
2. **Usability Requirements:**
    - The admin interface should feature clear menus, analytics panels, and visual indicators for low stock or pending orders.
    - All buttons and controls must be consistently labeled.

3. **Reliability Requirements:**

   - Product and order data must remain consistent across the database and frontend views.
   - Order status updates should trigger automatic notifications to relevant customers.
   - Daily backups of vendor data shall be performed automatically.

4. **Security Requirements:**

   - Only authenticated vendors may access the dashboard.
   - Vendor accounts must have role-based permissions (e.g., Add Product, Update Order).
   - Sensitive actions like deleting items or confirming delivery require confirmation prompts.

5. **Maintainability and Scalability:**

   - The system must be modular, allowing easy addition of new features (analytics, reports).
   - The database must support an expanding product catalog and user base.
   - All vendor actions should be logged for auditing and debugging purposes.

6. Constraints

- Only authorized vendors can access this module.
- Offline functionality is not supported
- The module depends on the central CUET Online Stationery database for all operations.
- Hosting and deployment will use university or free-tier cloud infrastructure with limited resources.
- Development must be completed within the academic semester, following phased delivery (core → advanced features).

7. Assumptions and Dependencies

- The central database will remain synchronized with both Customer and Vendor modules.
- Vendors will maintain accurate product details and stock quantities.
- Delivery staff and logistics support will handle completed orders.
- System maintenance will be carried out by authorized personnel only.
- All third-party tools (chart libraries, APIs) will remain available and compatible.

8. Future Enhancements

1. **Automated Restock Alerts:**
   The system could notify vendors automatically when item quantities reach a predefined minimum.
2. **Sales Analytics Dashboard:**
   Integration of graphical analytics for daily, weekly, and monthly sales performance tracking.
3. **Vendor Performance Evaluation:**
   Reports to analyze delivery efficiency, stock management, and overall order accuracy.
4. **Multi-Vendor Marketplace Expansion:**
   Expand the platform to include multiple campus vendors with separate dashboards.

9. Conclusion

The Vendor Module of the CUET Online Stationery System enables efficient product and order management, ensuring students receive timely service while vendors can maintain organized inventory and streamlined operations across the campus.

3.4 Development of Use Case Template

A use case template provides a structured and detailed textual description of each use case identified in the use case diagram. It documents the step-by-step interactions between actors and the system, including normal flows, alternative flows, and exception scenarios. The template ensures consistency in documenting functional requirements and serves as a reference for system design and testing phases.

3.4.1 Customer Use Case Template

Product Search Functionality

Use case: Search Products

Primary actor: Customer

Goal in context: To allow customers to find specific stationery items by name, category, or keyword in the CUET Online Stationery system.

Preconditions: Customer must be logged into the system and connected to the internet.

Trigger: Customer selects the search option and enters a product name or keyword.

Scenario:

1. Customer selects the search bar.
2. Customer enters a keyword or product name.
3. System displays matching items from the product catalog.

Exceptions:

1. No matching product found. System will display a "No items found" message.
2. Network issue; search results cannot be retrieved.

Priority: Essential, must be implemented.

When available: First increment.

Frequency of use: Many times per session.

Channel to actor: Website or mobile interface.

Secondary actors: Database system.

Channels to secondary actors: Database queries via system API.

Open issues:

1. Should search allow partial matches or only exact matches?
2. Should there be filters for price range, brand, or category?

Catalog Browsing

Use case: View Product Catalog

Primary actor: Customer

Goal in context: To enable customers to browse all available stationery items and view their details before ordering.

Preconditions: Customer is logged into the system and product data is available.

Trigger: Customer selects "View Catalog" from the main menu.

Scenario:

1. Customer clicks on "View Product Catalog."
2. System displays categorized lists of available products.
3. Customer can click on any product to view details such as price, stock, and vendor.

Exceptions:

1. Catalog data unavailable; system shows maintenance message.
2. Product details temporarily inaccessible due to vendor updates.

Priority: Essential.

When available: First increment.

Frequency of use: Frequently used by customers.

Channel to actor: Website or mobile interface.

Secondary actors: Vendor system, product database.

Channels to secondary actors: Database connection and vendor APIs.

Open issues:

1. Should the catalog auto-update in real time when vendors change stock?
2. Should customers be allowed to mark favorites for quick access?

Order Placement

Use case: Place Order

Primary actor: Customer

Goal in context: To enable customers to place an order for selected stationery items.

Preconditions: Customer has selected at least one item and added it to the cart.

Trigger: Customer clicks on "Place Order" or proceeds to checkout.

Scenario:

1. Customer reviews cart items.
2. Customer confirms delivery details.
3. System verifies item availability and places the order.

4. Confirmation message is shown to the customer.

Exceptions:

1. Item out of stock; system prompts customer to remove it.
2. Server error during checkout.

Priority: Essential.
When available: Second increment.
Frequency of use: Once per order.
Channel to actor: Website or mobile interface.
Secondary actors: Vendor system, database, payment service.
Channels to secondary actors: API and order management interface.
Open issues:

1. Should the system allow multiple vendors in one order?
2. Should order cancellation be possible before payment?

Payment Processing

Use case: Make Payment
Primary actor: Customer
Goal in context: To complete the payment for the placed order using available methods.
Preconditions: Customer has a confirmed order ready for payment.
Trigger: Customer clicks "Proceed to Payment."
Scenario:

1. System displays payment options.
2. Customer selects preferred payment method.
3. Customer completes the transaction.
4. System verifies payment and updates order status to "Paid."

Exceptions:

1. Payment declined or interrupted.
2. Payment gateway temporarily unavailable.

Priority: Essential.

When available: Second increment.

Frequency of use: Once per order.

Channel to actor: Online payment interface.

Secondary actors: Payment gateway, bank systems.

Channels to secondary actors: Secure API for transaction verification.

Open issues:

1. Should there be an option for cash-on-delivery?
2. Should partial payments be allowed?

Order Tracking

Use case: Track Order Status

Primary actor: Customer

Goal in context: To allow customers to view the current status of their order (e.g., pending, processed, delivered).

Preconditions: Customer must have at least one active or completed order.

Trigger: Customer selects "Track Order Status."

Scenario:

1. Customer opens the order tracking page.
2. System retrieves current order status from the database.
3. Status and delivery updates are displayed to the customer.

Exceptions:

1. Server delay; tracking information not updated.
2. Order ID invalid or missing.

Priority: Essential.

When available: Final increment.

Frequency of use: Occasionally after placing orders.

Channel to actor: Website or mobile interface.

Secondary actors: Vendor system, database.

Channels to secondary actors: Data synchronization via API.

Open issues:

1. Should notifications be sent automatically when the status changes?

2. Should customers be allowed to contact vendors directly through the tracking page?

3.4.2 Vendor Use Case Template

Update Product Catalog

Use case: Update Product Catalog

Primary actor: Vendor

Goal in context: To allow vendors to add, edit, or remove product information such as name, price, and description in the CUET Online Stationery system.

Preconditions: Vendor must be logged into the system and have permission to modify catalog data.

Trigger: Vendor selects "Manage Catalog" option.

Scenario:

1. Vendor accesses the catalog management section.

2. Vendor adds a new product or selects one to update.

3. Vendor edits product details such as price or description.

4. System validates and updates the product catalog.

Exceptions:

1. Invalid or missing data — system prompts correction.

2. Network or database failure during update.

Priority: Essential.

When available: First increment.

Frequency of use: Occasionally, when stock or pricing changes.

Channel to actor: Vendor dashboard.

Secondary actors: Product database.

Channels to secondary actors: Database updates via system API.

Open issues:

1. Should bulk product updates be supported?

2. Should vendors be allowed to upload product images directly?

Inventory Monitoring

Use case: Monitor Inventory Level

Primary actor: Vendor

Goal in context: To enable vendors to track current stock quantities and receive alerts when items are low in stock.

Preconditions: Vendor has added products to the catalog and stock data is recorded.

Trigger: Vendor opens the "Inventory Dashboard."

Scenario:

1. Vendor accesses the inventory monitoring section.
2. System displays available quantities for each product.
3. Vendor identifies low-stock items and decides whether to restock.

Exceptions:

1. Stock data unavailable due to sync error.
2. Incorrect stock count shown due to delayed update.

Priority: High.

When available: Second increment.

Frequency of use: Daily.

Channel to actor: Vendor dashboard.

Secondary actors: Database system.

Channels to secondary actors: Automated data synchronization via API.

Open issues:

1. Should system automatically alert vendors when stock is low?
2. Should inventory auto-update after each confirmed order?

Order Viewing

Use case: View Orders

Primary actor: Vendor

Goal in context: To allow vendors to view all customer orders related to their products, including order status and details.

Preconditions: Vendor is logged in and customer orders exist in the system.

Trigger: Vendor selects “View Orders” from the main dashboard.

Scenario:

1. Vendor opens the order management section.
2. System displays all orders placed for the vendor’s products.
3. Vendor reviews order details such as customer name, items, and total amount.

Exceptions:

1. No orders found — system displays “No current orders.”
2. Temporary database issue — orders cannot be loaded.

Priority: Essential.

When available: Second increment.

Frequency of use: Several times a day.

Channel to actor: Vendor dashboard.

Secondary actors: Order management system, database.

Channels to secondary actors: Database query through API.

Open issues:

1. Should orders be filterable by date, payment, or delivery status?
2. Should vendors be notified in real-time when new orders arrive?

Order Processing

Use case: Process Orders

Primary actor: Vendor

Goal in context: To enable vendors to confirm, package, and dispatch customer orders efficiently.

Preconditions: Vendor has at least one pending order ready for processing.

Trigger: Vendor selects an order to process.

Scenario:

1. Vendor reviews the pending order list.
2. Vendor confirms stock availability.
3. Vendor packages items and marks the order as “Ready for Dispatch.”

4. System updates the order status and notifies the customer.

Exceptions:

1. Product out of stock; order placed on hold.
2. Technical issue while updating status.

Priority: High.
When available: Final increment.
Frequency of use: Daily or as orders arrive.
Channel to actor: Vendor dashboard.
Secondary actors: Order database, customer notification system.
Channels to secondary actors: API and notification services.
Open issues:

1. Should vendors be able to print invoices directly from the dashboard?
2. Should automatic delivery tracking be integrated?

3.5 Activity Diagram

The activity diagram illustrates the dynamic behavior of the system by modeling the flow of control between different activities. It represents how users interact with the system and how various processes are executed from start to finish.

In the activity diagram of our CUET Online Stationery System, two main actors are involved: the Student and the Vendor. After logging in, a student can browse products using search and filter options, view details, and add items to the cart. Once the order is placed, the student selects a payment method, either bKash or Cash on Delivery or may choose to cancel the order. When payment is confirmed, the student can track the order status and provide feedback upon receiving the order.

On the vendor's side, after login, the vendor accesses the Vendor Dashboard to manage products. They can add, edit, or remove items, monitor stock levels, and process incoming orders. The vendor also assigns deliveries and views reports on sales and customer feedback to improve service and performance.

Overall, the activity diagram captures the workflow of both users and vendors, showing the smooth transaction process, order management, and interaction between different system activities.

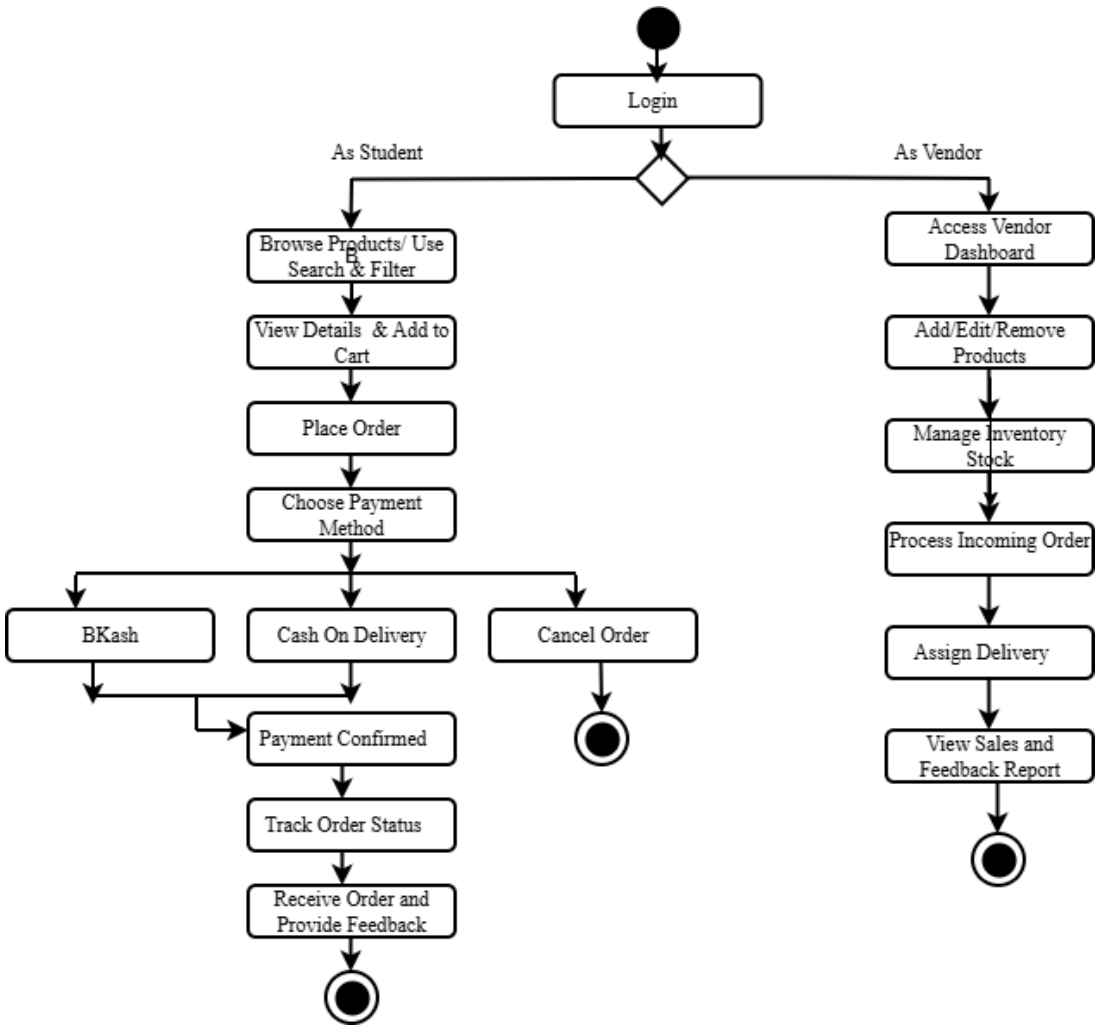


Figure 3: Activity Diagram

3.6 Static Model – Class Diagram

The class diagram is a representation that portrays the system in a static and structural manner, providing a visual depiction of the relationships between classes and the attributes, functions, and associations of each class. Unlike dynamic diagrams that depict the interactions between components, the class diagram focuses on the static structure of the system. In the class diagram of our online stationary store system, the Customer class serves as the central element, linked to the Order class which stores information about purchases made by customers. The Order class has a compositional relationship with the OrderItem class, which contains detailed information about individual products within each order. Each OrderItem refers to a specific Product through an association relationship. The Product class is connected to the Vendor class, indicating that products are supplied by vendors. The Payment class is established to handle transaction processing. Additionally, the Recommender class analyzes customer purchases and suggests relevant products, enhancing the shopping experience.

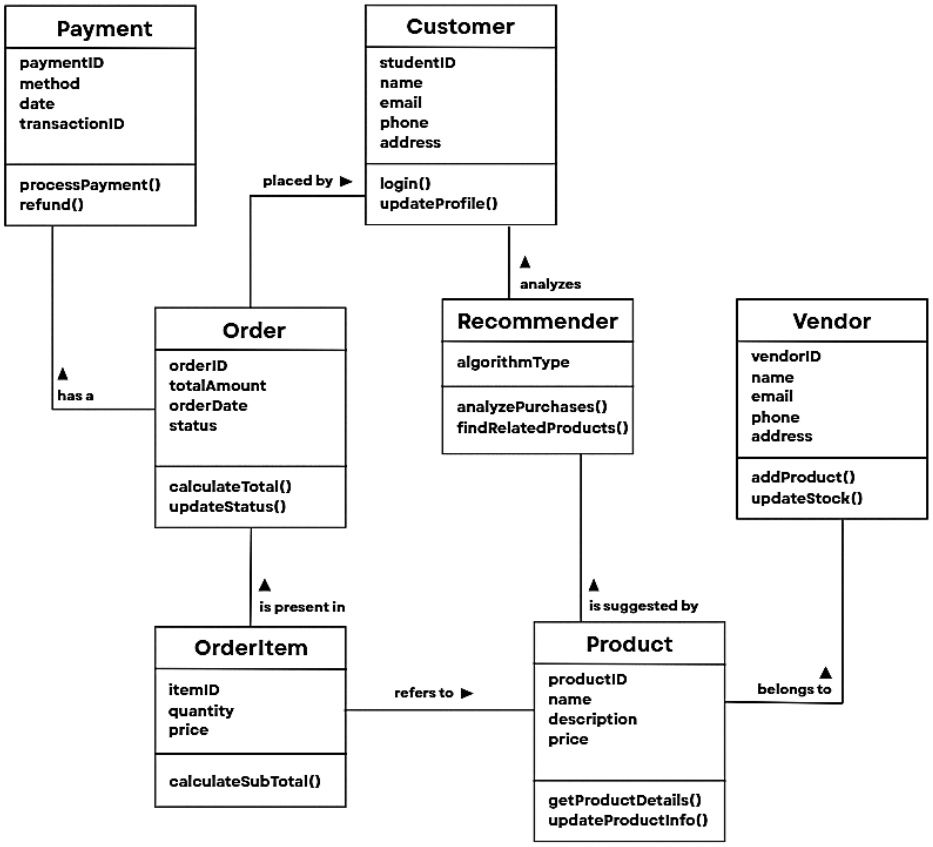


Figure 4: Class Diagram

3.7 Dynamic Model – Sequence Diagram

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that shows how objects interact with each other in a particular sequence of time. It focuses on the order of messages or events exchanged between different system components to carry out a specific process or function.

In a sequence diagram, objects or actors are represented by vertical lines called lifelines, and the messages they exchange are shown as horizontal arrows. The diagram helps visualize the flow of control, the timing of interactions, and the logic of a scenario within a system. This first diagram illustrates the interactions between the Student, System, and Database during the login process and while searching for stationery items. It shows how the user credentials are verified and how product information is retrieved from the database for display. This second diagram represents the sequence of interactions that occur when a student selects items, adds them to the cart, confirms the order, and completes payment. It also depicts how the system communicates with the Payment Gateway, Database, and Vendor to process the order and manage delivery updates.

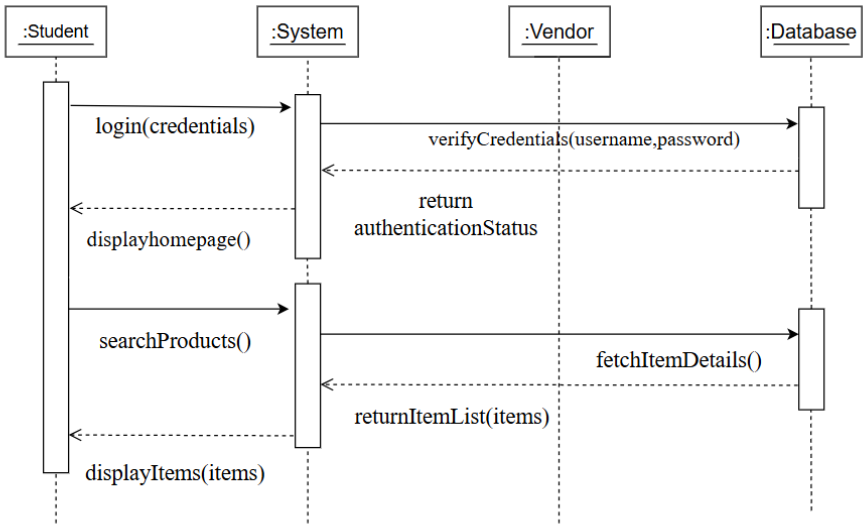


Figure 5: Sequence Diagram for Login and Product Search

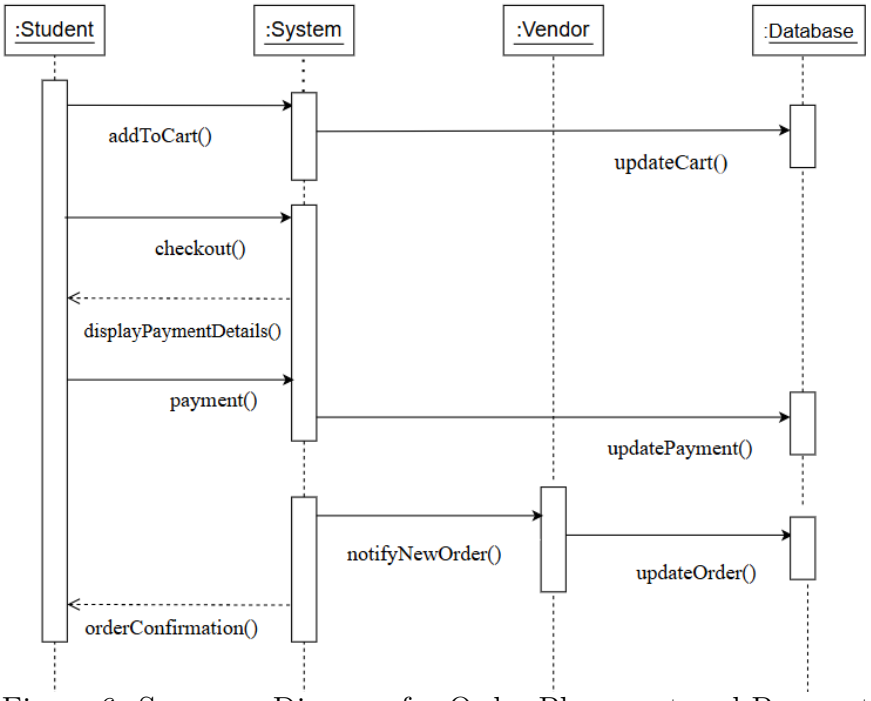


Figure 6: Sequence Diagram for Order Placement and Payment

3.8 Safety and Security Requirements

3.8.1 Access Requirements

- Only registered CUET students can access the platform using their student ID and password.
- Admins and vendors have separate login credentials with role-based access control to manage products, inventory, and orders.
- Unauthorized users attempting to access restricted areas will be automatically redirected to the login page.

- Multi-level authentication (e.g., one-time password via email or phone) may be implemented for additional security during payment or account recovery.

3.8.2 Integrity Requirements

- All order details, including item name, quantity, price, and delivery information, must be stored accurately and remain unaltered.
- Only authorized admins can update or delete product and inventory information; students can only view these details.
- The system must prevent any unauthorized modification of order or payment data once confirmed.
- All admin actions, such as adding or removing items, should be logged with timestamps for accountability.
- Regular database backups must be performed to protect against data loss due to system failures or unexpected shutdowns.

3.8.3 Privacy Requirements

- Personal information (such as name, hall address, contact number, and payment details) will be stored securely and used only for order processing and delivery purposes.
- The system will comply with data protection standards and ensure end-to-end encryption for sensitive information during transactions.
- User data will not be shared with third parties without explicit consent.
- Upon account deletion or project completion, all personal data will be permanently removed from the database.

4 Architectural Design

4.1 Architectural Model/Style Used

For the CUET Online Stationery System, a **data-centered architectural model** has been adopted with a primary focus on centralized data management, consistency, and reliable information sharing among system components.

In this design, a **centralized data repository** acts as the core of the system. All major system components—including customers, vendors, and administrators—interact indirectly through this shared data store. The repository maintains product catalogs, user credentials, order details, payment records, and delivery information, ensuring that all system operations are based on a single, consistent source of data.

The client-side modules (**Customer Module** and **Vendor Module**) access and update information by communicating with the centralized database via the system services. Customers can browse products, place orders, and complete payments, while vendors update inventory levels, manage product listings, and confirm order fulfillment. Each action performed by any component is reflected immediately in the shared data repository, maintaining real-time synchronization across the system.

This data-centered architecture improves data integrity, system reliability, and ease of maintenance, making it well suited for continuous and coordinated operation within the CUET campus environment.

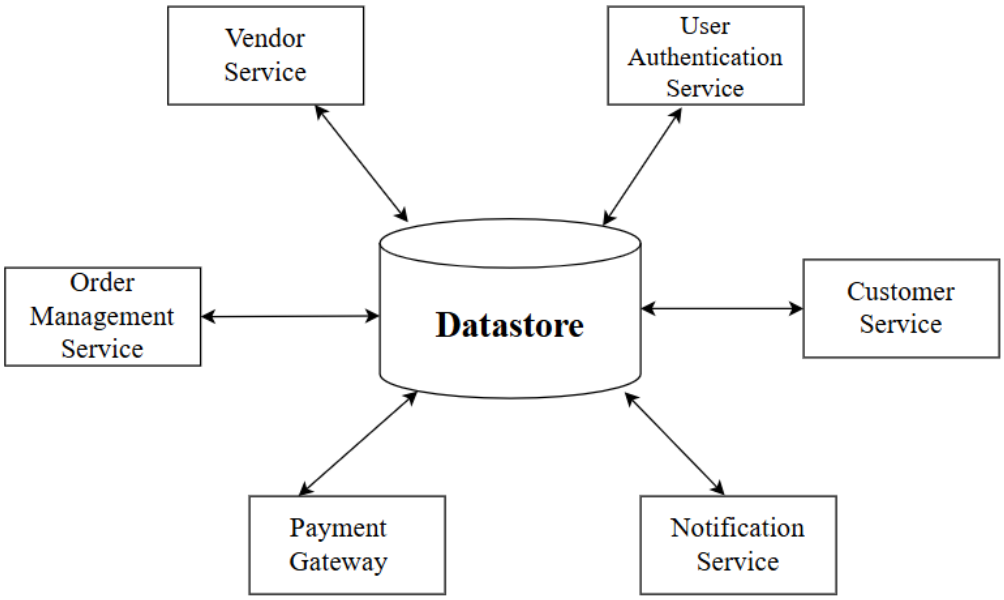


Figure 7: Data-Centered Architectural Model of CUET Online Stationery System

4.2 Rationale for Choosing the Architectural Model/Style

The selection of a **data-centered architectural model** for the CUET Online Stationery System is motivated by the need for strong data consistency, centralized control, and efficient information sharing.

This model enables:

- Vendors to manage product information, inventory updates, and order status through a single shared data source.
- Customers to access up-to-date product availability, place orders, make payments, and track delivery status accurately.
- Administrators to monitor system activities, manage users, and maintain overall data integrity.

By centralizing all critical data in one repository, the system minimizes redundancy and ensures that all components operate on accurate and consistent information. The data-centered approach also supports future system expansion, allowing features such as analytics, reporting tools, or integration with the university's ERP system to be added with minimal impact on existing components.

Overall, this architecture provides a stable, scalable, and data-reliable foundation, ensuring efficient service delivery for CUET students and vendors.

5 Design

5.1 Structured Chart Analysis

In the CUET Online Stationery System structure chart, the architectural design is decomposed into clear functional modules that follow a top-down refinement approach. The root module, CUET Online Stationery System, delegates major responsibilities to five primary submodules: *User Management*, *Product Management*, *Order Processing*, and *Payment & AI Services*.

Each of these modules is further refined into lower-level functions tailored to specific user roles, ensuring a clear separation of concerns between Students and Vendors. For instance, *Product Management* is split into *Vendor: Update Inventory* and *Student: Browse Catalog*, while *Order Processing* distinguishes between *Student: Place Order* and *Vendor: Manage Order Status*.

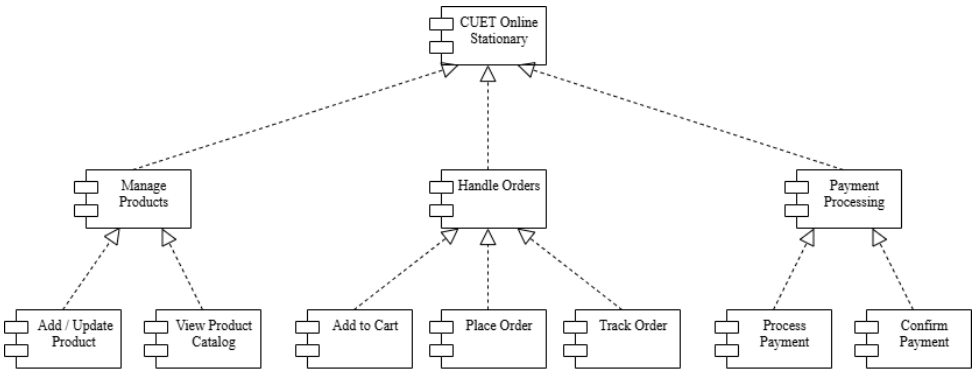


Figure 8: Structure Chart for CUET Online Stationery System

This hierarchical decomposition illustrates the functional independence of each module and emphasizes low coupling and high cohesion. The structure chart visually represents how different parts of the system collaborate—integrating shared services like *Process Payment* and *AI Recommendations*—while keeping operations modular, replaceable, and aligned with structured design principles.

5.2 Component Level Design

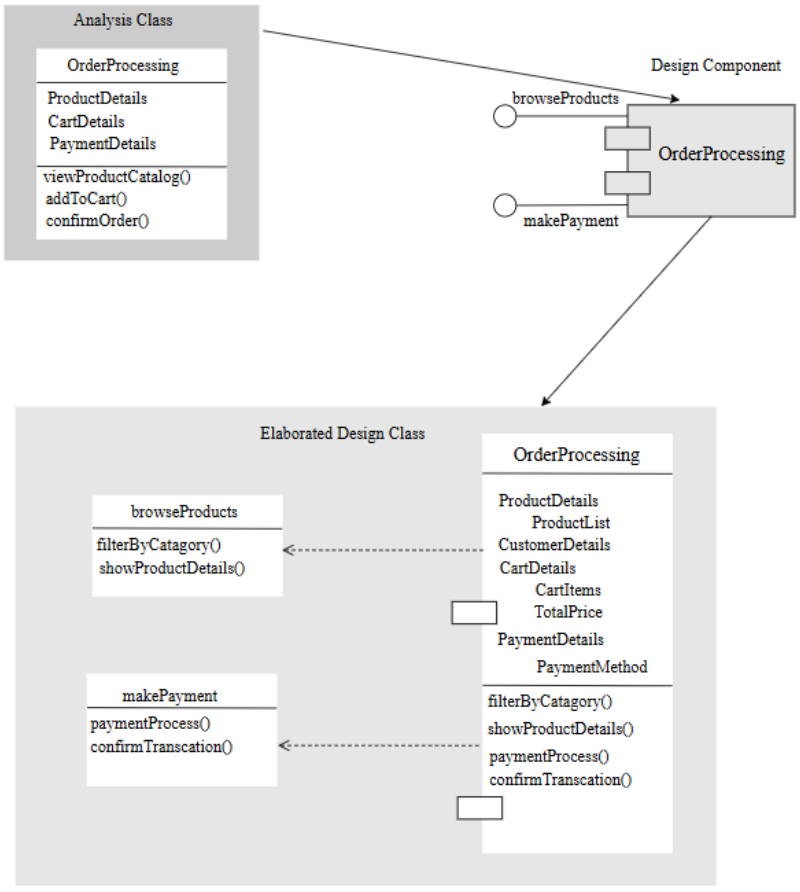


Figure 9: Component Level Design Diagram of Customer Service of CUET Online Stationery System

The component-level design diagram illustrates the transition from analysis to design by detailing how the OrderProcessing component is structured and interacts with its internal elements. The analysis class defines key data entities such as product, cart, and payment details, along with the main operations required for order management.

At the design level, the component exposes interfaces like browseProducts and makePayment, which correspond to user interactions. These interfaces are further elaborated into design classes handling specific responsibilities such as filtering products, displaying product information, processing payments, and confirming transactions.

This modular decomposition ensures better maintainability, scalability, and clear separation of functionalities within the system.

5.3 Technical Design Document

5.3.1 Automation (Cronjobs, Queue, etc.)

Cron jobs are used to automate scheduled tasks that need to run at regular intervals without user involvement. These tasks can handle recurring operations such as checking inventory, sending notifications, and generating reports. In the CUET Online Stationery API, cron jobs help automate routine tasks that ensure the system remains updated and efficient.

Cron-based automation includes:

- **Inventory Updates:** Automatically update stock levels based on orders and restocking events.
- **Email Notifications:** Send automated reminders for order confirmations, product promotions, or back-in-stock alerts.
- **Sales Report Generation:** Generate daily, weekly, or monthly sales summaries for administrative review.
- **System Maintenance:** Clean up outdated logs or expired data to maintain system performance.

Cron-based scheduling ensures timely execution of critical background tasks with minimal overhead, helping the system maintain smooth operations without manual intervention.

5.3.2 Queue System (Asynchronous Processing)

Task queues are used to manage time-consuming processes by running them asynchronously in the background. These queues allow tasks such as payment processing, product feedback analysis, or report generation to be handled without interrupting the main application or affecting the user experience.

Queues are implemented to handle tasks efficiently:

- **Order Processing:** After an order is placed, tasks like updating stock, processing payments, and generating invoices are queued for background processing.
- **Product Feedback:** When feedback is submitted, it is placed in a queue for analysis and categorization without delaying the user's interaction.
- **Large Report Generation:** Generate sales or inventory reports in the background, ensuring real-time interactions with the application remain unaffected.

Using a queue system improves the system's scalability by distributing heavy workloads across background workers, ensuring the application remains fast, responsive, and efficient even during high usage periods.

5.4 GUI (Graphical User Interface) Design

The GUI design of CUET Online Stationery System follows a user-centered approach with intuitive navigation, clean layouts, and responsive design. The interface is divided into two main modules: Student Interface and Admin Interface.

5.4.1 Home Page and Login

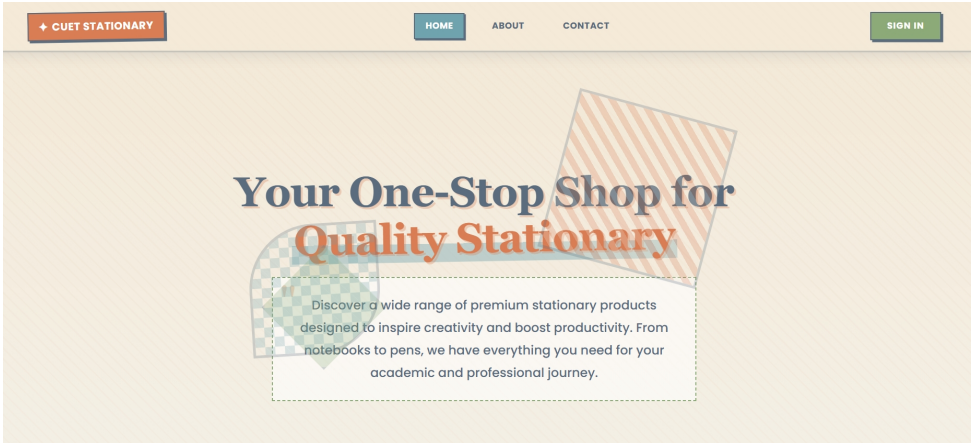


Figure 10: Home Page - Main Landing Page

The home page provides an overview of the stationery platform with featured products, categories, and quick access to login/registration. It includes navigation to all major sections and promotional banners.

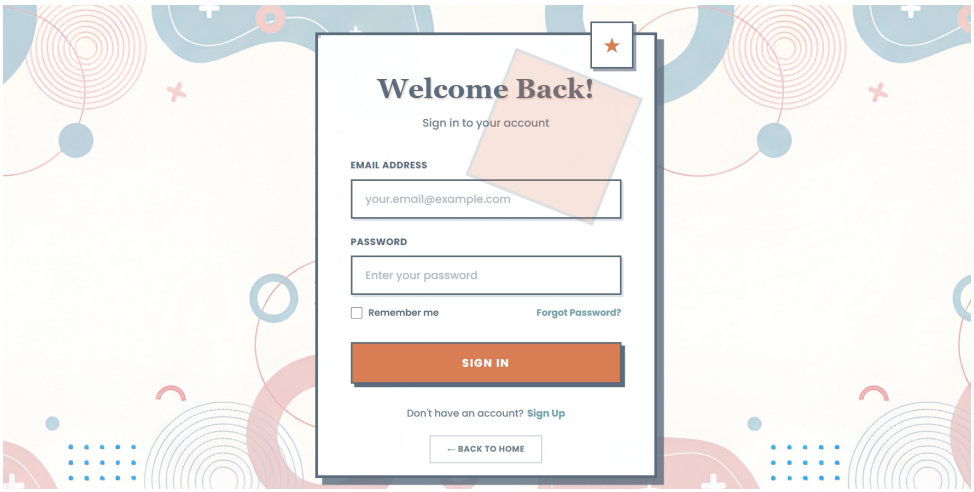


Figure 11: Login Page - User Authentication

The login page allows users to authenticate using their student credentials. It includes options for password recovery and new user registration with proper validation.

5.4.2 Student Interface

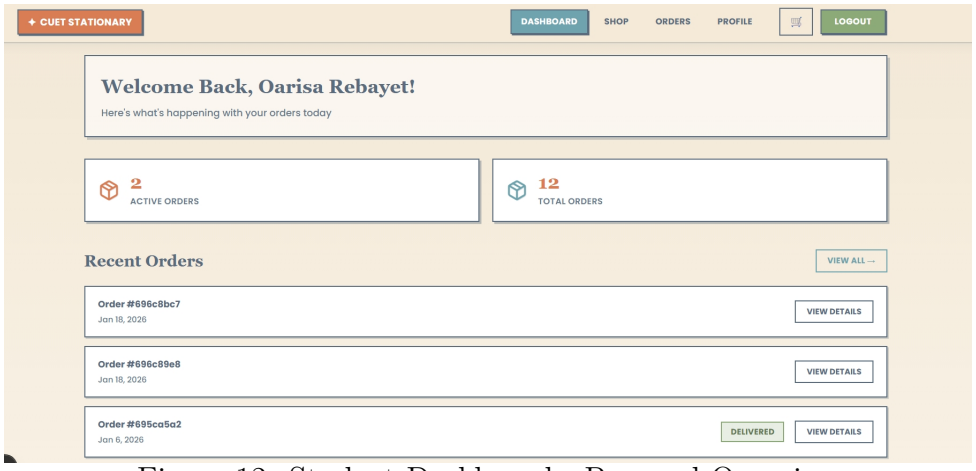


Figure 12: Student Dashboard - Personal Overview

The student dashboard displays personalized information including recent orders, recommended products, and account summary. It serves as the central hub for student activities.

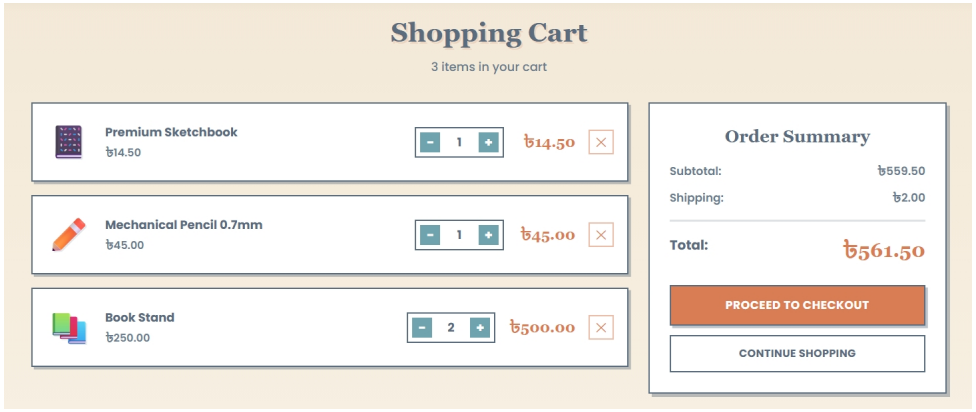


Figure 13: Shopping Cart - Order Management

The shopping cart interface shows selected items with quantities, prices, and total amount. Students can modify quantities, remove items, and proceed to checkout from this page.

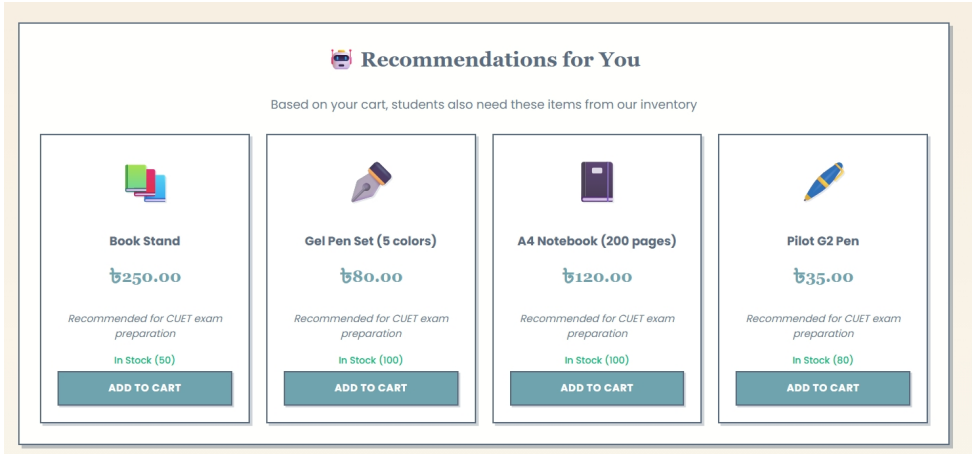


Figure 14: AI Recommendations - Smart Suggestions

The AI recommendation section provides personalized product suggestions based on browsing history, purchase patterns, and popular items among CUET students.

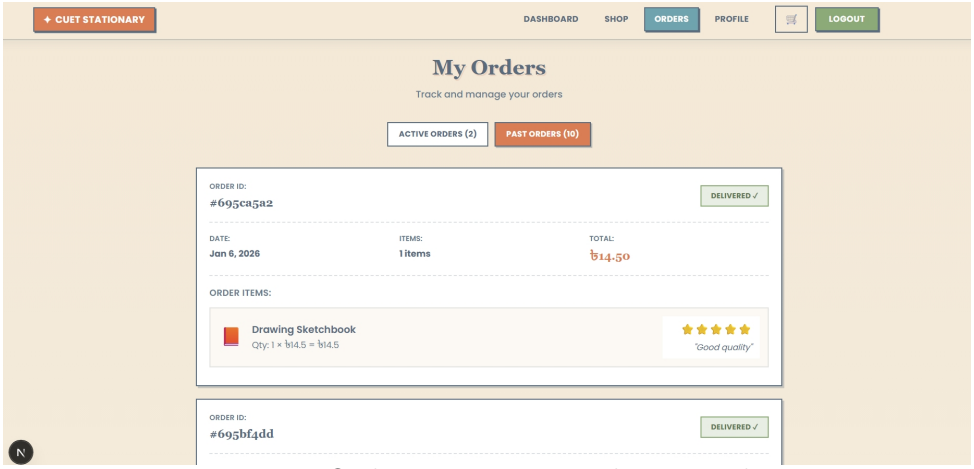


Figure 15: Order History - Purchase Tracking

The order history page displays all past and current orders with detailed status information. Students can track deliveries and view order details from this interface.

5.4.3 Admin Interface

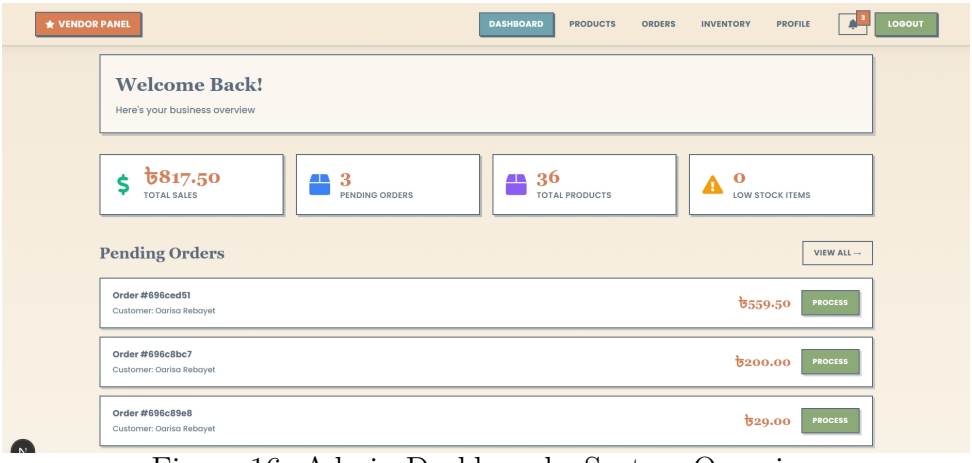


Figure 16: Admin Dashboard - System Overview

The admin dashboard provides a comprehensive overview of system performance including sales statistics, inventory levels, and recent orders for quick management.

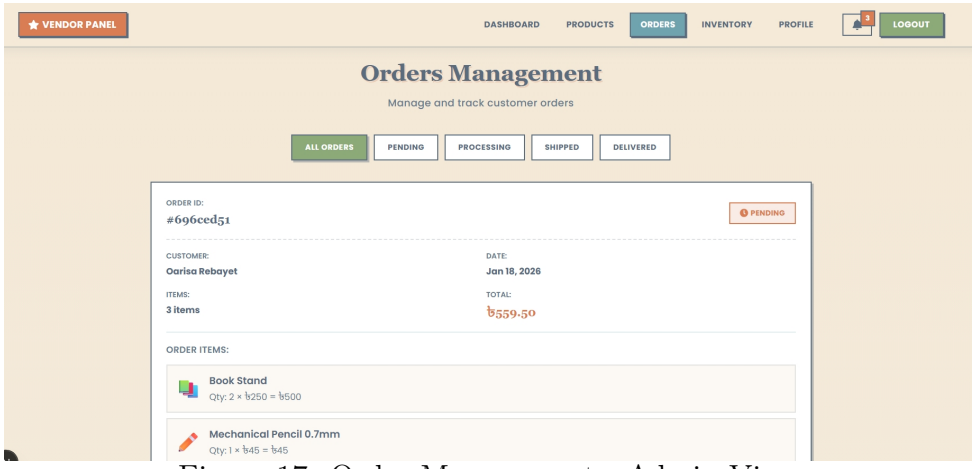


Figure 17: Order Management - Admin View

The order management interface allows administrators to view, process, and update order statuses. It includes filters for different order states and batch operations.

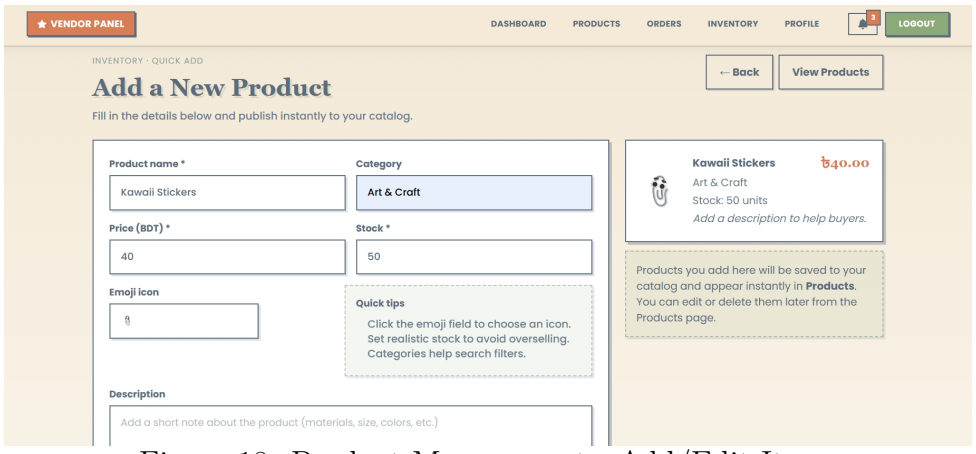


Figure 18: Product Management - Add/Edit Items

The product management page enables administrators to add new products, update existing items, and manage product categories and pricing information efficiently.

5.5 Data Flow Diagram

A graphical representation of the flow of data within a system is called a data flow diagram (DFD). It shows how data moves between external entities, data repositories, and processes. Data stores show where data is stored, external entities identify sources or destinations of data, processes represent actions or transformations, and arrows show the direction of data flow.

5.5.1 First Level DFD of the System

The first level DFD (Context Diagram) shows the overall system boundary and its interactions with external entities

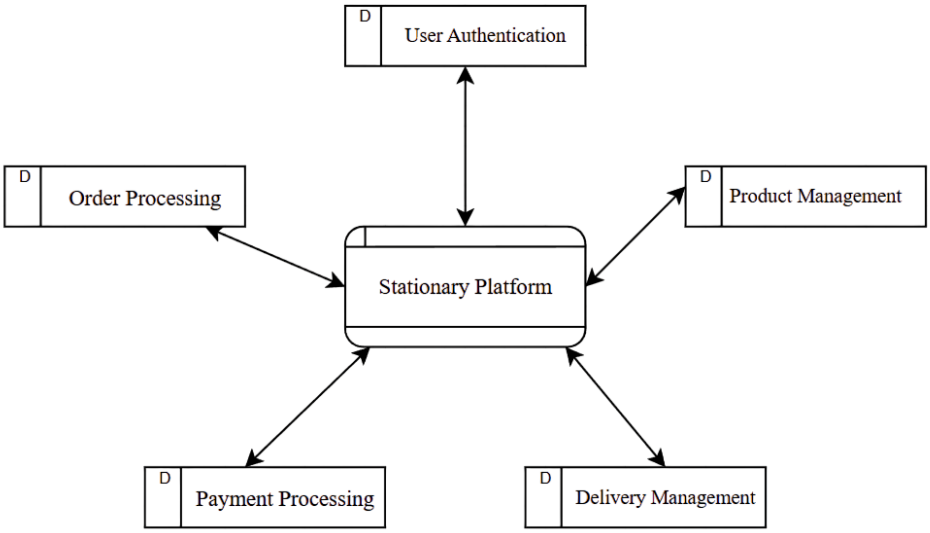


Figure 19: First Level DFD - Context Diagram of CUET Online Stationery System

5.5.2 User DFD

The User DFD (Level 1 DFD) details the data flows and processes from the student's perspective. It breaks down the main system into key processes that students interact with directly.

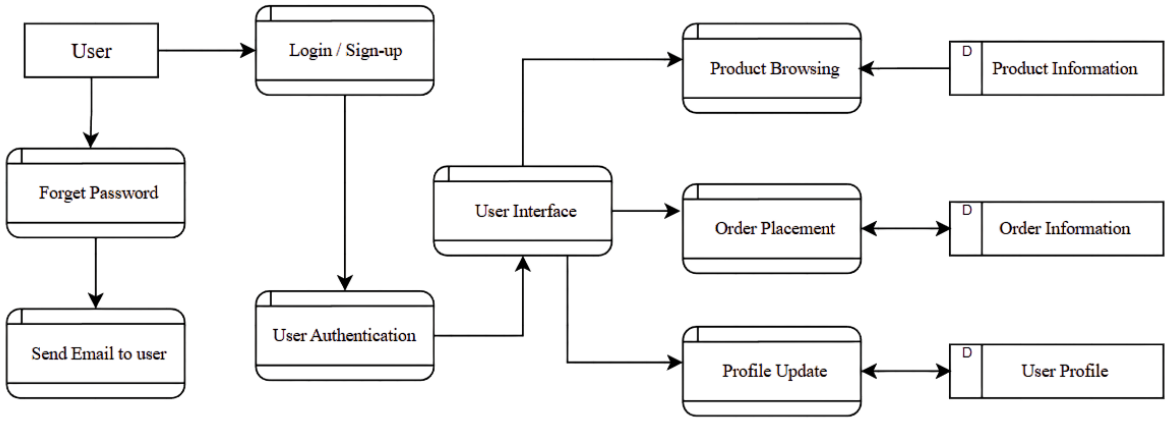


Figure 20: User DFD - Student Perspective

5.5.3 Admin DFD

The Admin DFD details the data flows and processes from the vendor/admin perspective. It shows how administrators manage the backend operations of the system.

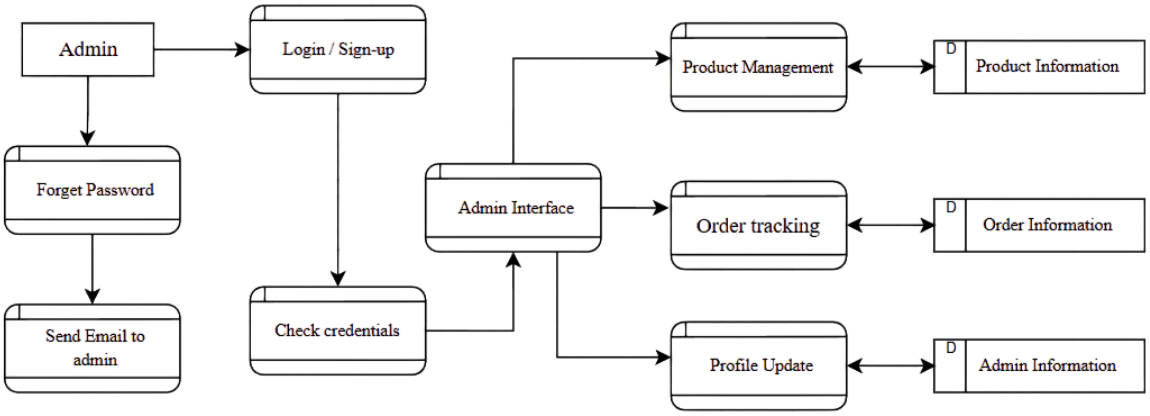


Figure 21: Admin DFD - Vendor/Admin Perspective

The DFDs collectively illustrate how the CUET Online Stationery System efficiently manages data flow from initial user interactions, such as registration and product browsing, through order placement, payment processing, and delivery coordination, up to comprehensive administrative management. These diagrams demonstrate the systematic transformation, storage, and retrieval of data across various system modules. Together, they emphasize the system's ability to ensure seamless information flow, minimize data redundancy and maintain high standards of data integrity, consistency, and security.

5.5.4 API Documentation

1. Create a new user account

Endpoint: `POST /auth/signup`

Description: Create a new user account with email, password, and personal information.

Request Body:

```
{
    "id": 1,
    "name": "Dipannita Paul Orni",
    "email": "u2104125@student.cuet.ac.bd",
    "password": "SecurePass123!",
    "studentId": "2104125",
```

```
    "phone": "+8801712345678",
    "address": "Chittagong, Bangladesh"
}
```

Response:

```
{
    "success": true,
    "message": "User registered successfully",
    "data": {
        "userId": 1,
        "name": "Dipannita Paul Orni",
        "email": "u2104125@student.cuet.ac.bd",
        "studentId": "2021-1-60-125"
    }
}
```

2. Login to get access token Endpoint: `POST /auth/login`

Description: Authenticate user and receive an access token for API authorization.

Request Body:

```
{
    "id": 2,
    "email": "u2104129@student.cuet.ac.bd",
    "password": "SecurePass123!"
}
```

Response:

```
{
    "success": true,
    "message": "Login successful",
    "data": {
        "token": "eyJhbGciOiJIUzI1NiIsInR...",
        "user": {
            "userId": 2,
            "name": "Oarisa Rebayet",
            "email": "u2104129@student.cuet.ac.bd",
```

```
            "studentId": "2021-1-60-129"
        }
    }
}
```


3. Get all products

Endpoint: `GET /products`

Description: Retrieve all available stationery products with details including price, stock, and category.

Response:


```
{
    "success": true,
    "data": [
        {
            "productId": "prod_001",
            "name": "A4 Notebook",
            "category": "Notebooks",
            "price": 150.00,
            "stock": 50,
            "description": "High quality ruled notebook"
        },
        {
            "productId": "prod_002",
            "name": "Blue Pen",
            "category": "Pens",
            "price": 20.00,
            "stock": 200,
            "description": "Smooth writing ballpoint pen"
        }
    ]
}
```


4. Create a new order

Endpoint: `POST /orders`

Description: Create a new order with product items, quantities, and delivery details.

Request Body:

```
{
    "items": [
        {
            "productId": "prod_001",
            "quantity": 2,
            "price": 150.00
        },
        {
            "productId": "prod_002",
            "quantity": 5,
            "price": 20.00
        }
    ],
    "deliveryAddress": "CUET Campus, Chittagong",
    "paymentMethod": "cash_on_delivery"
}
```

Response:

```
{
    "success": true,
    "message": "Order placed successfully",
    "data": {
        "orderId": "ORD_20240101_001",
        "totalAmount": 400.00,
        "status": "pending",
        "estimatedDelivery": "2024-01-03"
    }
}
```

5. Update product details

Endpoint: `PUT /admin/products/id`

Description: Update product information including name, price, stock, and category. Admin access only.

Request Body:

```
{
```

```
    "name": "Premium A4 Notebook",
    "price": 180.00,
    "stock": 75,
    "category": "Notebooks",
    "description": "Premium quality ruled notebook with hard cover"
}
```

Response:

```
{
    "success": true,
    "message": "Product updated successfully",
    "data": {
        "productId": "prod_001",
        "name": "Premium A4 Notebook",
        "price": 180.00
    }
}
```

6. Delete a product

Endpoint: `DELETE /admin/products/id`

Description: Delete a product from the inventory. Admin access only.

Response:

```
{
    "success": true,
    "message": "Product deleted successfully",
    "data": {
        "productId": "prod_001",
        "deletedAt": "2024-01-01T12:00:00Z"
    }
}
```

7. Submit product feedback

Endpoint: `POST /feedback`

Description: Submit product feedback and rating after purchase.

Request Body:

```
{
    "productId": "prod_001",
    "orderId": "ORD_20240101_001",
    "rating": 5,
    "comment": "Excellent quality notebook. Paper quality is great!",
    "userName": "Dipannita Paul Orni"
}
```


Response:


```
{
    "success": true,
    "message": "Feedback submitted successfully",
    "data": {
        "feedbackId": "FB_001",
        "productId": "prod_001",
        "rating": 5,
        "submittedAt": "2024-01-05T10:30:00Z"
    }
}
```


5.5.5 Swagger UI

The Swagger UI is an interactive tool that allows users to make API calls directly from the documentation. The UI for the CUET Online Stationery API is shown below:

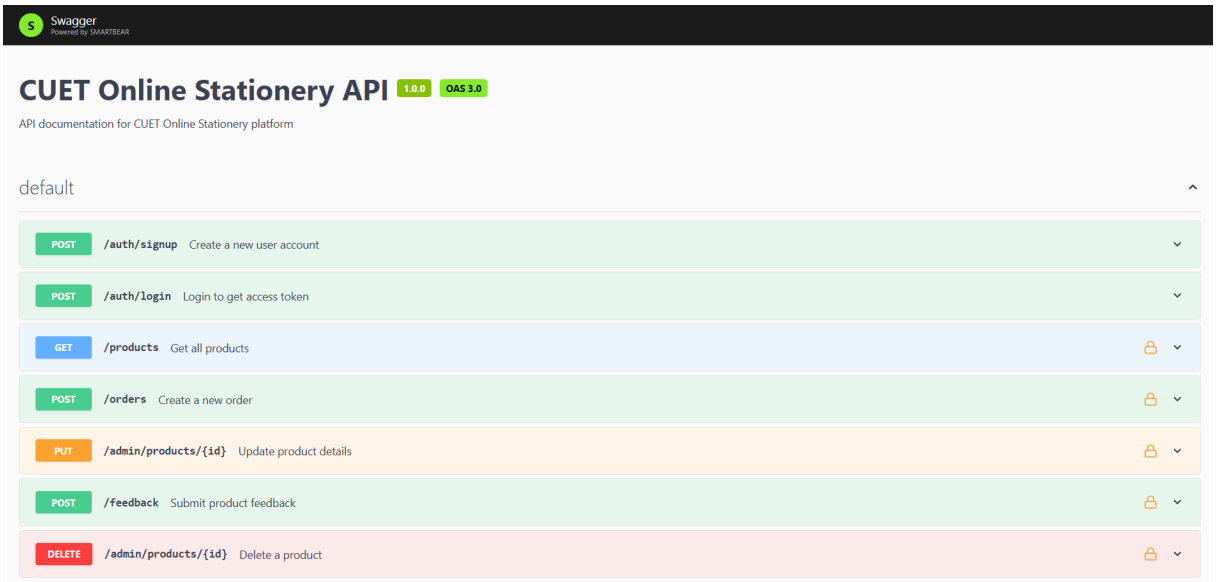


Figure 22: Swagger API Documentation Interface

5.5.6 Endpoints in Swagger

API 1: `POST /auth/signup` - Create a new user account.

API 2: `POST /auth/login` - Login to get access token.

API 3: `GET /products` - Get all products.

API 4: `POST /orders` - Create a new order.

API 5: `PUT /admin/products/id` - Update product details.

API 6: `POST /feedback` - Submit product feedback.

API 7: `DELETE /admin/products/id` - Delete a product.

1. **POST /auth/signup** - This image shows the Swagger UI for creating a new user account. It allows the user to input their personal details for registration.



Figure 23: Swagger UI for POST /auth/signup

2. **POST /auth/login** - This image shows the Swagger UI for logging in, where users can input their credentials (email and password) to obtain an access token.



Figure 24: Swagger UI for POST /auth/login

3. **GET /products** - This image shows the Swagger UI for fetching all products available in the stationery store, displaying product details like name, price, and stock.

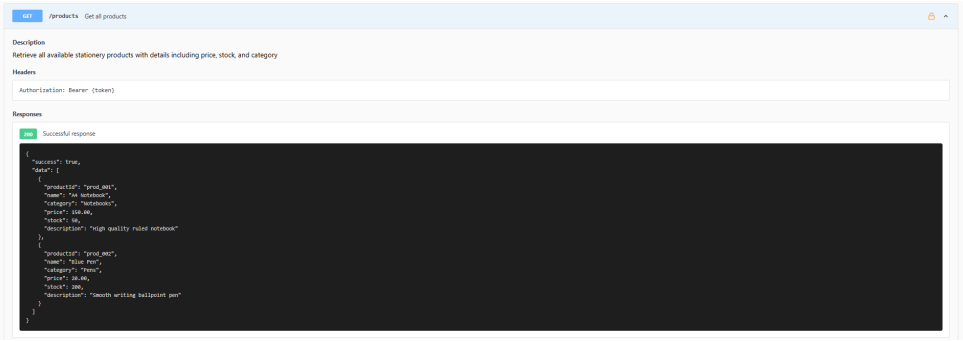


Figure 25: Swagger UI for GET /products

4. POST /orders - This image shows the Swagger UI for placing a new order, where users can specify the products, quantities, and delivery details.

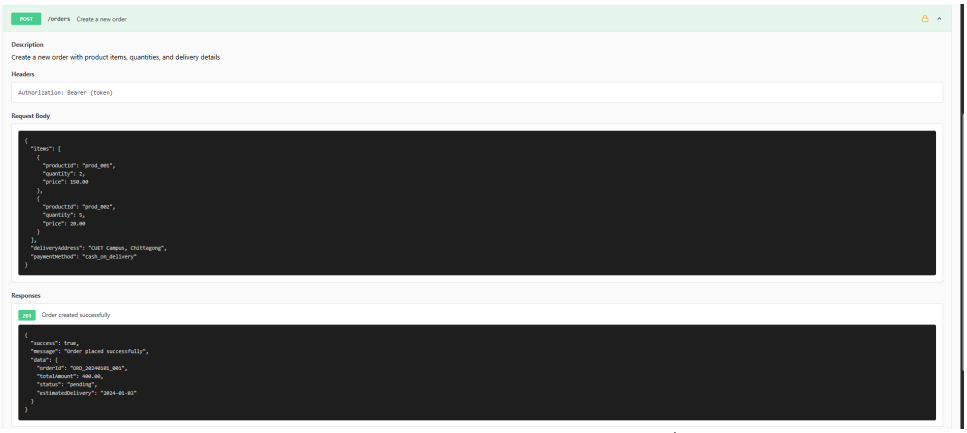


Figure 26: Swagger UI for POST /orders

5. PUT /admin/products/id - This image shows the Swagger UI for updating product details like name, price, and stock. Only accessible by admin.

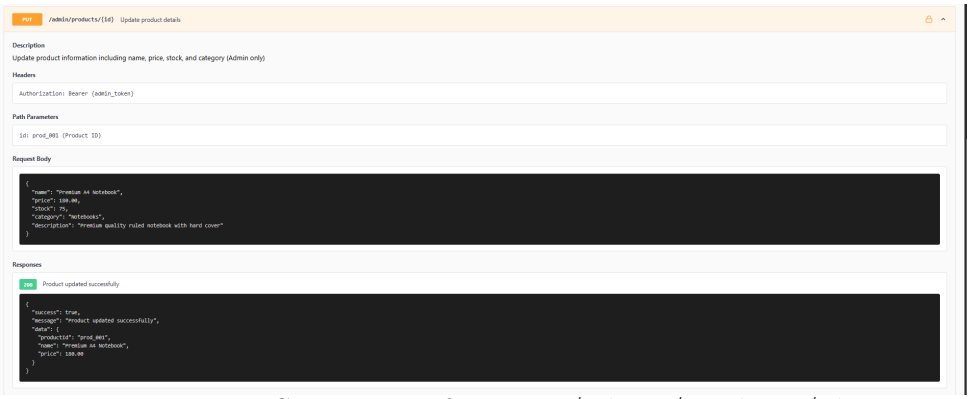


Figure 27: Swagger UI for PUT /admin/products/id

6. POST /feedback - This image shows the Swagger UI for submitting feedback on a product after purchase, allowing users to rate and comment.

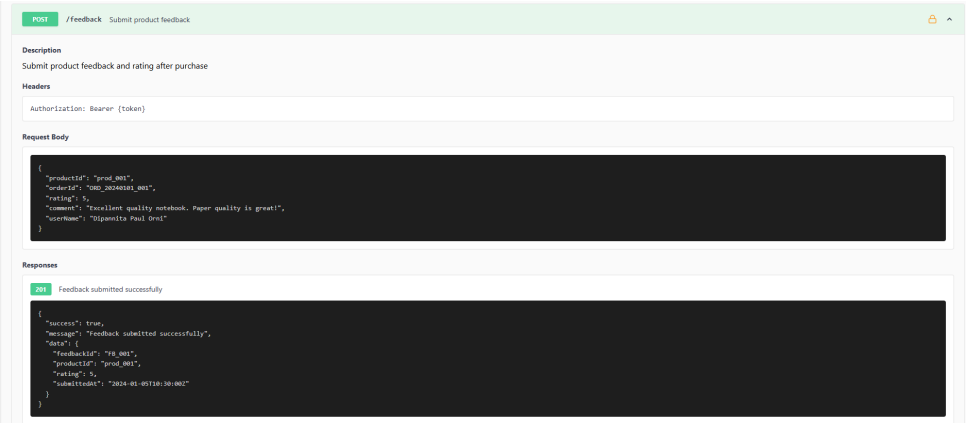


Figure 28: Swagger UI for POST /feedback

7. DELETE /admin/products/id - This image shows the Swagger UI for deleting a product from the inventory, accessible by admin only.



Figure 29: Swagger UI for DELETE /admin/products/id

5.6 ER Diagram

The ER diagram represents the database structure of the **CUET Online Stationery System**. It illustrates the entities, their attributes, and the relationships among students, admins, products, and transactions.

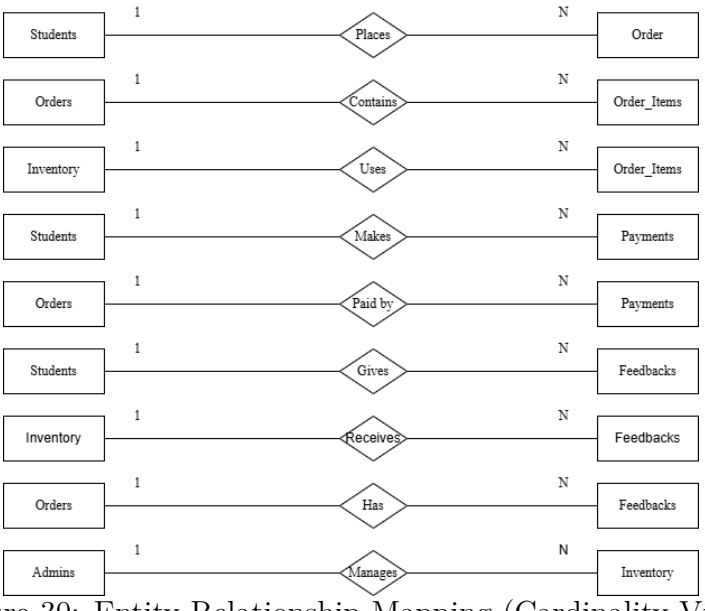


Figure 30: Entity Relationship Mapping (Cardinality View)

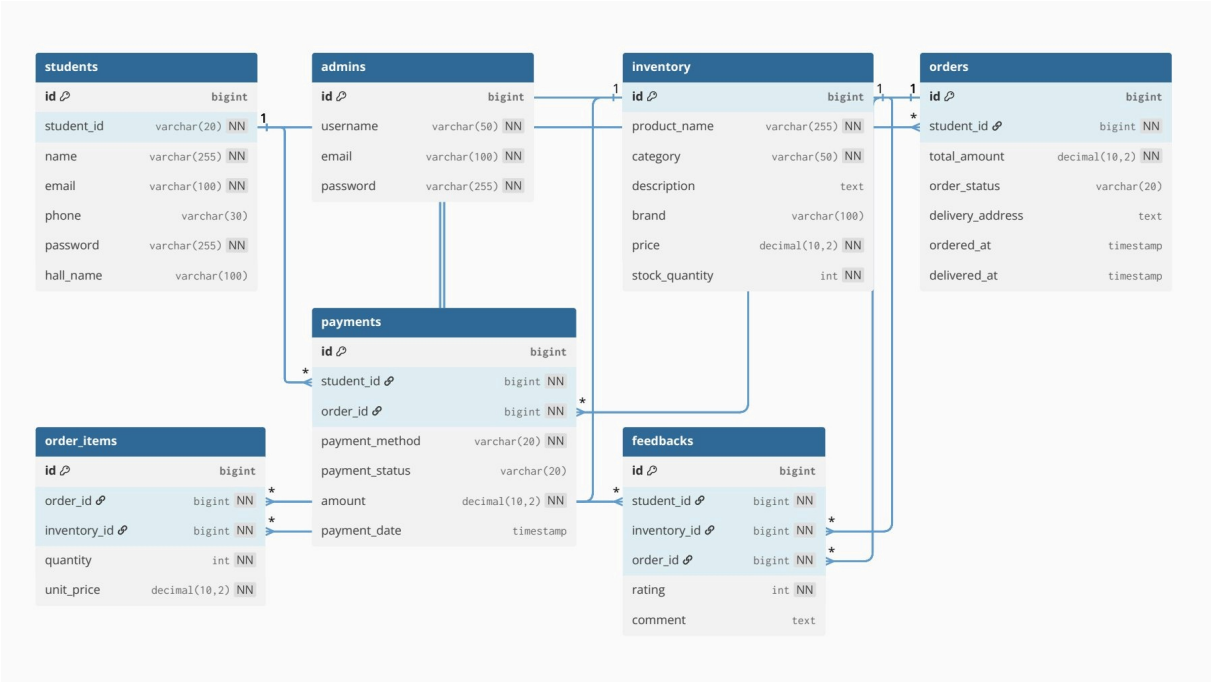


Figure 31: ER Diagram for CUET Online Stationery System

Entities and Attributes:

- **Students:** id, student_id, name, email, phone, password, hall_name
- **Admins:** id, username, email, password
- **Inventory:** id, product_name, category, description, brand, price, stock_quantity
- **Orders:** id, student_id, total_amount, order_status, delivery_address, ordered_at, delivered_at
- **Order_Items:** id, order_id, inventory_id, quantity, unit_price
- **Payments:** id, student_id, order_id, payment_method, payment_status, amount, payment_date
- **Feedbacks:** id, student_id, inventory_id, order_id, rating, comment

Relationship Overview:

- Students place Orders
- Orders contain Order_Items
- Order_Items use Inventory items
- Students make Payments

- Orders are paid by Payments
- Students give Feedbacks
- Inventory receives Feedbacks
- Orders have Feedbacks
- Admins manage Inventory

The relationships enable a complete workflow: students place orders containing inventory items, process payments, and provide feedback on their purchases.

5.7 Database Design Model and Script

The database is implemented using MongoDB, a NoSQL document-oriented database, integrated with Next.js full-stack framework. Mongoose ODM (Object Data Modeling) library is used to define schemas and manage data validation. This combination provides flexibility, scalability, and seamless integration with the JavaScript ecosystem.

5.7.1 Collection Descriptions

Students Collection The `students` collection stores information about registered students who can place orders through the system. Each student document contains authentication credentials and contact information.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Unique Fields:** `student_id`, `email`
- **Attributes:** student ID, name, email, phone, password (hashed), hall name
- **References:** Referenced by orders, payments, and feedbacks collections

Admins Collection The `admins` collection maintains administrative user accounts with elevated privileges for system management and oversight.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Unique Fields:** `username`, `email`
- **Attributes:** username, email, password (hashed)

Inventory Collection The `inventory` collection catalogs all products available for purchase, including detailed product information and stock levels.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Attributes:** product name, category, description, brand, price, stock quantity
- **References:** Referenced by order items and feedbacks collections

Orders Collection The `orders` collection records all purchase orders placed by students, tracking order amounts, status, and delivery information.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Foreign Reference:** `student_id` references `students` collection
- **Attributes:** student ID, total amount, order status, delivery address, order timestamp, delivery timestamp
- **References:** Referenced by order items, payments, and feedbacks collections

Order Items Collection The `order_items` collection stores line-item details for each order, linking orders with inventory items.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Foreign References:**
  - `order_id` references `orders` collection
  - `inventory_id` references `inventory` collection
- **Attributes:** order ID, inventory ID, quantity, unit price

Payments Collection The `payments` collection tracks all payment transactions associated with orders, supporting multiple payment methods and statuses.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Foreign References:**
  - `student_id` references `students` collection
  - `order_id` references `orders` collection
- **Attributes:** student ID, order ID, payment method, payment status, amount, payment timestamp

Feedbacks Collection The `feedbacks` collection stores customer reviews and ratings for products, enabling quality monitoring and customer satisfaction tracking.

- **Primary Identifier:** `_id` (ObjectId, auto-generated)
- **Foreign References:**
  - `student_id` references `students` collection
  - `inventory_id` references `inventory` collection
  - `order_id` references `orders` collection
- **Attributes:** student ID, inventory ID, order ID, rating (integer), comment (text)

5.7.2 Entity Relationships

The database implements a well-structured relationship model through ObjectId references, ensuring data consistency and enabling efficient queries across collections.

Relationship	**Description**
Students → Orders	One-to-Many: Each student can place multiple orders
Orders → Order Items	One-to-Many: Each order contains multiple line items
Inventory → Order Items	One-to-Many: Each product can appear in multiple orders
Students → Payments	One-to-Many: Tracks all payments made by a student
Orders → Payments	One-to-Many: Allows multiple payment transactions per order
Students → Feedbacks	One-to-Many: Students can submit multiple reviews
Inventory → Feedbacks	One-to-Many: Products receive multiple feedback entries
Orders → Feedbacks	One-to-Many: Links feedback to specific purchases

Table 5: Database Entity Relationships

5.7.3 Mongoose Schema Implementation

The following sections detail the Mongoose schema definitions for each collection. These schemas enforce data validation, establish relationships through references, and ensure consistency across the database.

Core User Schemas The system distinguishes between two types of users: students who place orders and administrators who manage the system. Both schemas implement secure authentication with hashed passwords and unique identifiers.

Listing 1: Student Schema Definition

```
import mongoose from 'mongoose';

const studentSchema = new mongoose.Schema({
  student_id: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  phone: {
    type: String,
    trim: true
  },
  password: {
    type: String,
    required: true
  },
  hall_name: {
    type: String,
    trim: true
```

```
  }
}, {
  timestamps: true
});

export default mongoose.models.Student ||
  mongoose.model('Student', studentSchema);
```

Listing 2: Admin Schema Definition

```
import mongoose from 'mongoose';

const adminSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true
  }
}, {
  timestamps: true
});

export default mongoose.models.Admin ||
  mongoose.model('Admin', adminSchema);
```

Product Management Schema The inventory schema manages all product-related information including pricing, stock levels, and categorization. Built-in validation ensures prices and quantities remain non-negative, preventing data inconsistencies.

Listing 3: Inventory Schema Definition

```
import mongoose from 'mongoose';

const inventorySchema = new mongoose.Schema({
  product_name: {
    type: String,
    required: true,
    trim: true
  },
  category: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    trim: true
  },
  brand: {
    type: String,
    trim: true
  },
  price: {
    type: Number,
    required: true,
    min: 0
  },
  stock_quantity: {
    type: Number,
    required: true,
    min: 0,
    default: 0
  }
```

```
}, {
  timestamps: true
});

export default mongoose.models.Inventory ||
  mongoose.model('Inventory', inventorySchema);
```

Order Processing Schemas The order management system uses two interconnected schemas. The main order schema tracks overall order information, while the order items schema maintains detailed line-item information for each product in an order.

Listing 4: Order Schema Definition

```
import mongoose from 'mongoose';

const orderSchema = new mongoose.Schema({
  student_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Student',
    required: true
  },
  total_amount: {
    type: Number,
    required: true,
    min: 0
  },
  order_status: {
    type: String,
    enum: ['pending', 'processing', 'shipped',
           'delivered', 'cancelled'],
    default: 'pending'
  },
  delivery_address: {
    type: String,
    required: true
  },
  ordered_at: {
    type: Date,
```

```
    default: Date.now
  },
  delivered_at: {
    type: Date
  }
}, {
  timestamps: true
});

export default mongoose.models.Order ||
  mongoose.model('Order', orderSchema);
```

Listing 5: Order Items Schema Definition

```
import mongoose from 'mongoose';

const orderItemSchema = new mongoose.Schema({
  order_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Order',
    required: true
  },
  inventory_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Inventory',
    required: true
  },
  quantity: {
    type: Number,
    required: true,
    min: 1
  },
  unit_price: {
    type: Number,
    required: true,
    min: 0
  }
}, {
```

```
  timestamps: true
});

export default mongoose.models.OrderItem ||
  mongoose.model('OrderItem', orderItemSchema);
```

Transaction Management Schema The payment schema handles financial transactions with support for multiple payment methods. Status tracking enables monitoring of payment lifecycles from initiation through completion or failure.

Listing 6: Payment Schema Definition

```
import mongoose from 'mongoose';

const paymentSchema = new mongoose.Schema({
  student_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Student',
    required: true
  },
  order_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Order',
    required: true
  },
  payment_method: {
    type: String,
    required: true,
    enum: ['cash', 'bkash']
  },
  payment_status: {
    type: String,
    enum: ['pending', 'completed'],
    default: 'pending'
  },
  amount: {
    type: Number,
    required: true,
```

```
    min: 0
  },
  payment_date: {
    type: Date
  }
}, {
  timestamps: true
});

export default mongoose.models.Payment ||
  mongoose.model('Payment', paymentSchema);
```

Customer Feedback Schema The feedback schema enables quality monitoring by collecting customer ratings and comments. This schema links feedback to specific students, products, and orders, providing comprehensive context for each review.

Listing 7: Feedback Schema Definition

```
import mongoose from 'mongoose';

const feedbackSchema = new mongoose.Schema({
  student_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Student',
    required: true
  },
  inventory_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Inventory',
    required: true
  },
  order_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Order',
    required: true
  },
  rating: {
    type: Number,
```

```
    required: true,
    min: 1,
    max: 5
  },
  comment: {
    type: String,
    trim: true
  }
}, {
  timestamps: true
});

export default mongoose.models.Feedback ||
  mongoose.model('Feedback', feedbackSchema);
```

6 Testing

Testing is the systematic process of evaluating a software system to ensure that its functions operate according to the specified requirements and behave reliably for the end user. It includes examining each module through component testing and validating the fully integrated system using system testing based on the requirement specification. This helps identify defects, verify functionality, and ensure consistent performance under various conditions. Along with manual checks, automation tools like Jenkins are used to run repetitive and critical tests efficiently, supporting continuous quality throughout development.

6.1 Manual Testing

Manual testing focuses on verifying the CUET Online Stationery system from an end-user perspective. These tests ensure that all functionalities work as expected when performed manually, including user authentication, product management, cart operations, order processing, and vendor operations. The following manual test cases cover critical scenarios of the system:

Project Name	CUET Online Stationery
Module Name	Shopping Cart - Add Product
Created By	Dipannita Paul Orni
Reviewed By	Oarisa Rebayet
Date of Creation	15-01-2026
Date of Review	16-01-2026

Test Case ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-CART-01	Verify add product to cart	1. Go to shop 2. Browse items 3. Select product 4. Click Add 5. Check cart	Product: Spiral Notebook Price: 45 BDT	Product added Cart updated Item visible	Added OK Cart shows 1 Item listed	Passed

Table 6: Test Case - Add Product to Cart

Project Name	CUET Online Stationery
Module Name	Vendor Product Management
Created By	Oarisa Rebayet
Reviewed By	Umme Sanjida
Date of Creation	1-01-2026
Date of Review	4-01-2026

Test Case ID	Scenario	Steps	Test Data	Expected Result	Actual Result	Status
TC-VENDOR-02	Verify vendor add product	1. Login vendor 2. Go Products 3. Add Product 4. Fill form 5. Save	Name: Graph Paper Price: 30 Stock: 200	Product saved Shows in list Visible in shop	Created OK In vendor list In shop	Passed

Table 7: Test Case - Vendor Add New Product

6.2 Automated Testing

Automated testing was implemented using Jenkins CI/CD pipeline to ensure code quality and functional correctness throughout the development lifecycle. The test suite was written in TypeScript using Jest testing framework. It covers critical functionalities including authentication, shopping cart operations, order management, and product handling.

6.2.1 Automated Test Cases

Project Name	CUET Online Stationery
Module Name	Authentication - Automated Tests
Test Suite	TC_AUTH_001, TC_AUTH_002
Automation Tool	Jenkins + Jest
Test Type	Unit Testing
Execution Date	Automated on each commit

Test Case ID	Test Description	Test Data	Expected Result	Status
TC_AUTH_ 01.1	Verify signup with valid data	Name: John Doe Email: john@example.com Pass: Test123456	Name true Email has @ Pass $\geq$ 6 chars	Passed
TC_AUTH_ 01.2	Verify signup fails with mismatch	Pass: Test123456 Confirm: Different123	Passwords mismatch Validation fails	Passed
TC_AUTH_ 01.3	Verify signup fails short pass	Password: 12345	Pass $<$ 6 chars Validation fails	Passed
TC_AUTH_ 02.1	Validate email format	Valid: test@example.com Invalid: invalid-email	Valid matches regex Invalid rejected	Passed
TC_AUTH_ 02.2	Verify password required	Email: test@example.com Password: (empty)	Pass field false Login fails	Passed

Table 8: Automated Test Cases - Authentication Module

Project Name	CUET Online Stationery
Module Name	Shopping Cart - Automated Tests
Test Suite	TC_CART_001, TC_CART_002
Automation Tool	Jenkins + Jest
Test Type	Unit Testing
Execution Date	Automated on each commit

Test Case ID	Test Description	Test Data	Expected Result	Status
TC_CART_ 01.1	Calculate total for single item	Product: Notebook Price: 50 Qty: 2	Total = $50 \times 2 = 100$	Passed
TC_CART_ 01.2	Calculate total for multiple items	Item 1: Notebook (50, Qty: 2) Item 2: Pen (10, Qty: 5)	Total = 150	Passed
TC_CART_ 01.3	Validate positive quantity	Quantity: 5	Qty $>$ 0 passes	Passed
TC_CART_ 02.1	Update cart quantity	Initial: 2 Updated: 5	Qty updates to 5	Passed

Table 9: Automated Test Cases - Shopping Cart Module

Project Name	CUET Online Stationery
Module Name	Orders - Automated Tests
Test Suite	TC_ORDER_001
Automation Tool	Jenkins + Jest
Test Type	Unit Testing
Execution Date	Automated on each commit

Test Case ID	**Test Description**	**Test Data**	**Expected Result**	**Status**
TC_ORDER_ 01.1	Validate order has items	Order: Notebook, Price: 50, Qty: 2	Order items > 0 Order is valid	Passed
TC_ORDER_ 01.2	Calculate order total	Item 1: Notebook (50, Qty: 2) Item 2: Pen (10, Qty: 3)	Total = 130	Passed

Table 10: Automated Test Cases - Orders Module

Project Name	CUET Online Stationery
Module Name	Products - Automated Tests
Test Suite	TC_PRODUCT_001, TC_PRODUCT_002
Automation Tool	Jenkins + Jest
Test Type	Unit Testing
Execution Date	Automated on each commit

Test Case ID	**Test Description**	**Test Data**	**Expected Result**	**Status**
TC_PRODUCT_ 01.1	Validate product fields	Name: Notebook Price: 50 Stock: 100	Fields truthy Price > 0 Stock ≥ 0	Passed
TC_PRODUCT_ 01.2	Prevent negative price	Price: -10	Price < 0 fails	Passed
TC_PRODUCT_ 01.3	Prevent negative stock	Stock: -5	Stock < 0 fails	Passed
TC_PRODUCT_ 02.1	Update stock correctly	Initial: 100 Restock: 50	Stock = 150	Passed
TC_PRODUCT_ 02.2	Detect low stock	Stock: 5 Threshold: 10	Low stock alert	Passed

Table 11: Automated Test Cases - Products Module

6.2.2 Automated Test Execution Summary

Test Suite	**Total Tests**	**Passed**	**Failed**	**Success Rate**
Authentication Tests	5	5	0	100%
Shopping Cart Tests	4	4	0	100%
Orders Tests	2	2	0	100%
Products Tests	5	5	0	100%
Overall	**16**	**16**	**0**	**100%**

Table 12: Automated Test Execution Summary

6.2.3 Jenkins CI/CD Integration

The automated test suite is integrated with Jenkins CI/CD pipeline, providing the following benefits:

- **Continuous Testing**: Tests execute automatically on each code commit

- **Early Bug Detection**: Issues identified immediately during development
- **Regression Prevention**: Ensures new changes don't break existing functionality
- **Quality Gates**: Code passes all tests before deployment
- **Test Reports**: Detailed execution logs and coverage reports generated

Figure 32 shows the Jenkins dashboard displaying a successful build execution. The automated pipeline validates code quality, runs all test suites, and confirms that all test cases pass successfully before deployment.

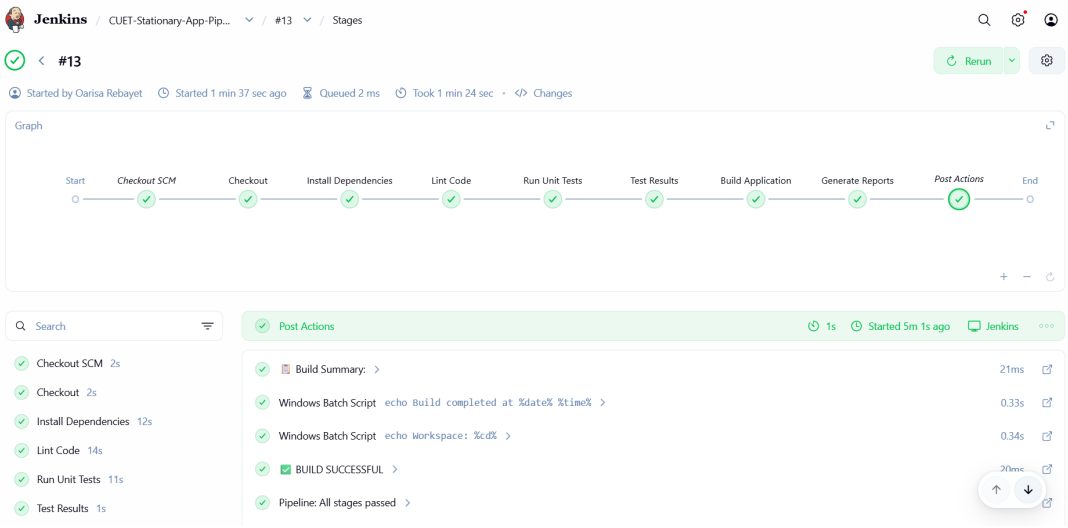


Figure 32: Jenkins Build and Test Execution - Successful Build

6.3 Techniques Used for Test Generation

In this project, both white box testing and black box testing techniques were applied to generate effective test cases for the CUET Online Stationery System. These techniques were selected to ensure comprehensive validation of both internal logic and external functionality of the system.

White Box Testing Technique: Basis Path Testing

The Basis Path Testing technique was used to analyze the internal control flow of critical modules such as authentication, order processing, and inventory management. This technique involves constructing flow graphs of the program logic and identifying independent execution paths. Test cases were derived to ensure that each statement and decision point in the code was executed at least once, thereby improving code coverage and reducing the possibility of logical errors.

Black Box Testing Technique: Equivalence Partitioning

Equivalence Partitioning was employed as the primary black box testing technique to

evaluate system functionality based on input conditions. Input data was divided into valid and invalid equivalence classes, such as correct and incorrect login credentials or valid and invalid product quantities. Test cases were designed from these partitions to verify that the system responds correctly to both acceptable and erroneous inputs without examining the internal code structure.

6.4 Assessment of the Goodness of the Test Suite

The testing phase of the CUET Online Stationery System followed a systematic and comprehensive approach to ensure software correctness, reliability, and usability. Both white box and black box testing methods were applied to cover different aspects of system validation.

White box testing using the Basis Path Testing technique enabled analysis of the logical complexity of internal modules and ensured adequate execution path coverage. This was particularly important due to the interaction between multiple components such as user authentication, product management, and order processing.

In addition, black box testing using Equivalence Partitioning focused on validating functional requirements from the user's perspective. By testing valid and invalid input conditions across various features, the test suite ensured that the system behaves as expected under different scenarios. The combined application of these testing techniques enhanced the robustness of the test suite and increased confidence in the system's overall reliability and performance.

6.5 Sustainability Plan

The sustainability of the CUET Online Stationery System is ensured through design choices that support long-term usability, maintainability, and growth. The sustainability plan emphasizes balanced development, focusing on system efficiency, ease of maintenance, and adaptability to future requirements. Rather than prioritizing rapid expansion, the system is designed to evolve steadily while maintaining stability and performance.

6.5.1 Scalability

Scalability is a key consideration in the design of the CUET Online Stationery System. The system follows a client-server architectural model with centralized data management, enabling it to support an increasing number of users without significant performance

degradation. The use of modular components and a centralized database allows the system to handle growing workloads, expanded product catalogs, and increased transaction volumes efficiently. This ensures that the platform remains responsive and reliable as user demand increases over time.

6.5.2 Flexibility and Customization

The system is designed with flexibility and ease of customization in mind. The modular structure of the web application allows individual components, such as product management or order tracking, to be modified or extended without affecting the entire system. This design approach enables administrators to adapt the system to changing user needs, introduce new features, or update existing functionalities with minimal effort. As a result, the CUET Online Stationery System remains adaptable and cost-effective throughout its lifecycle.

References

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.